

의료와 데이터사이언스 9 주차

비정형 데이터 생체신호 분석 1: 생체신호 전처리

Hyun-Lim Yang, PhD

Assistant Professor

Office of Hospital Information
Seoul National University Hospital

Adjunct Assistant Professor

Department of Medicine
Seoul National University



CONTENTS

Introductory – ECG and PPG

Signal data processing

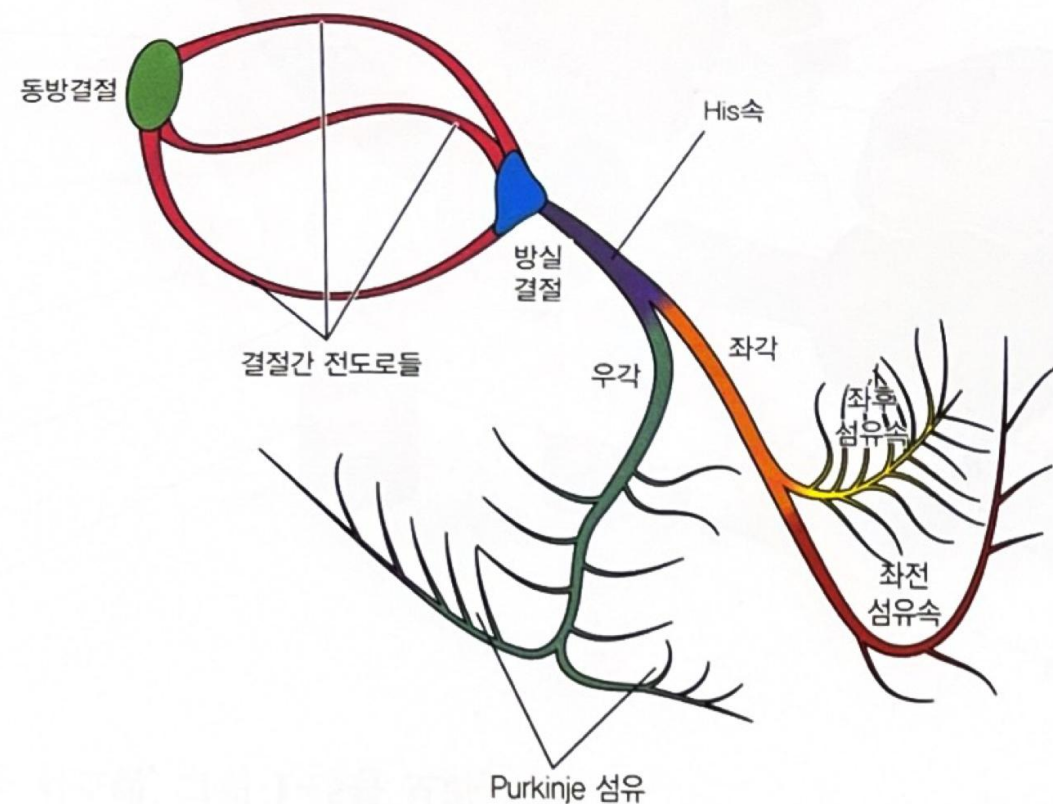
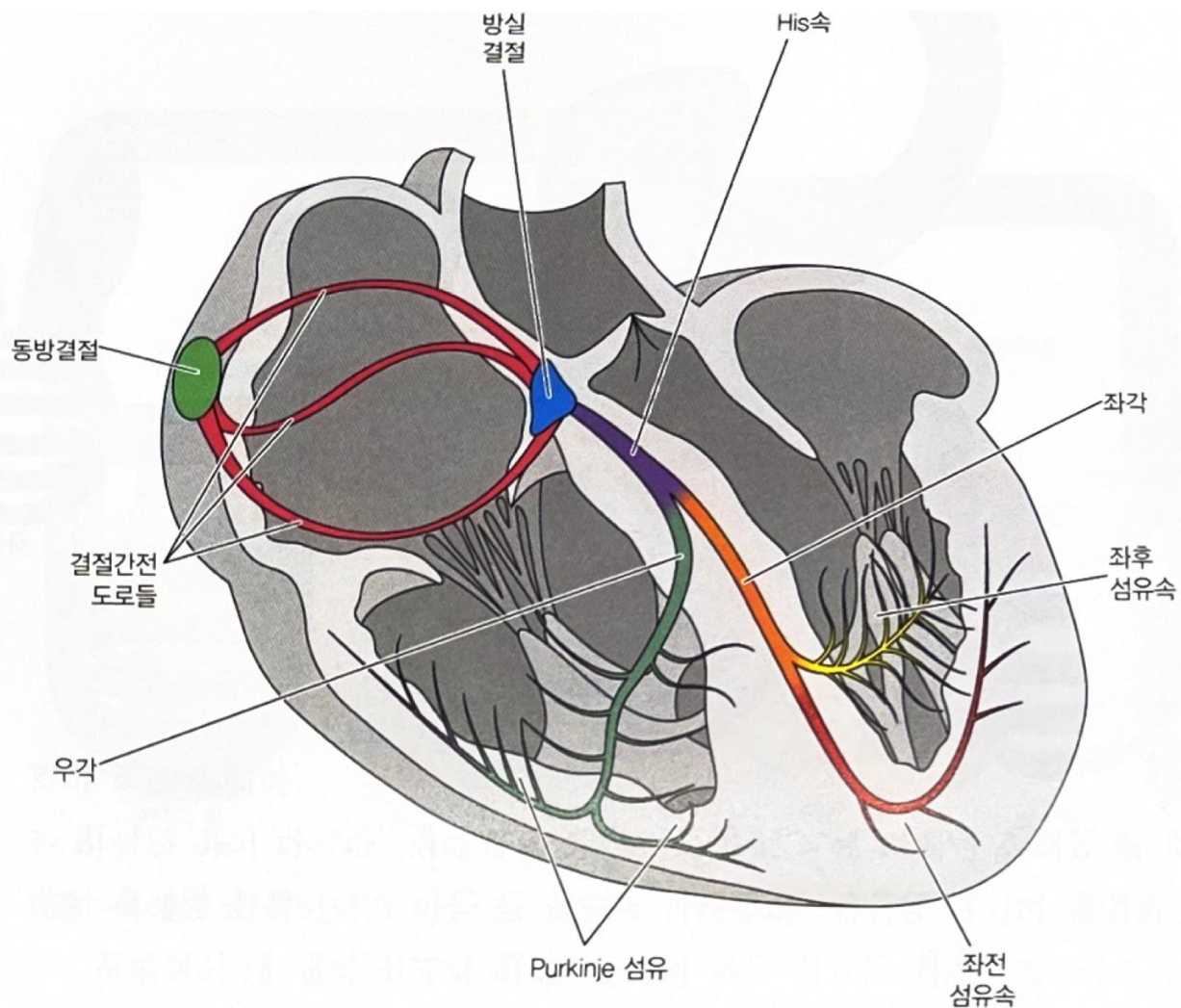
Signal data filtering

Hands-on: preprocessing for AI

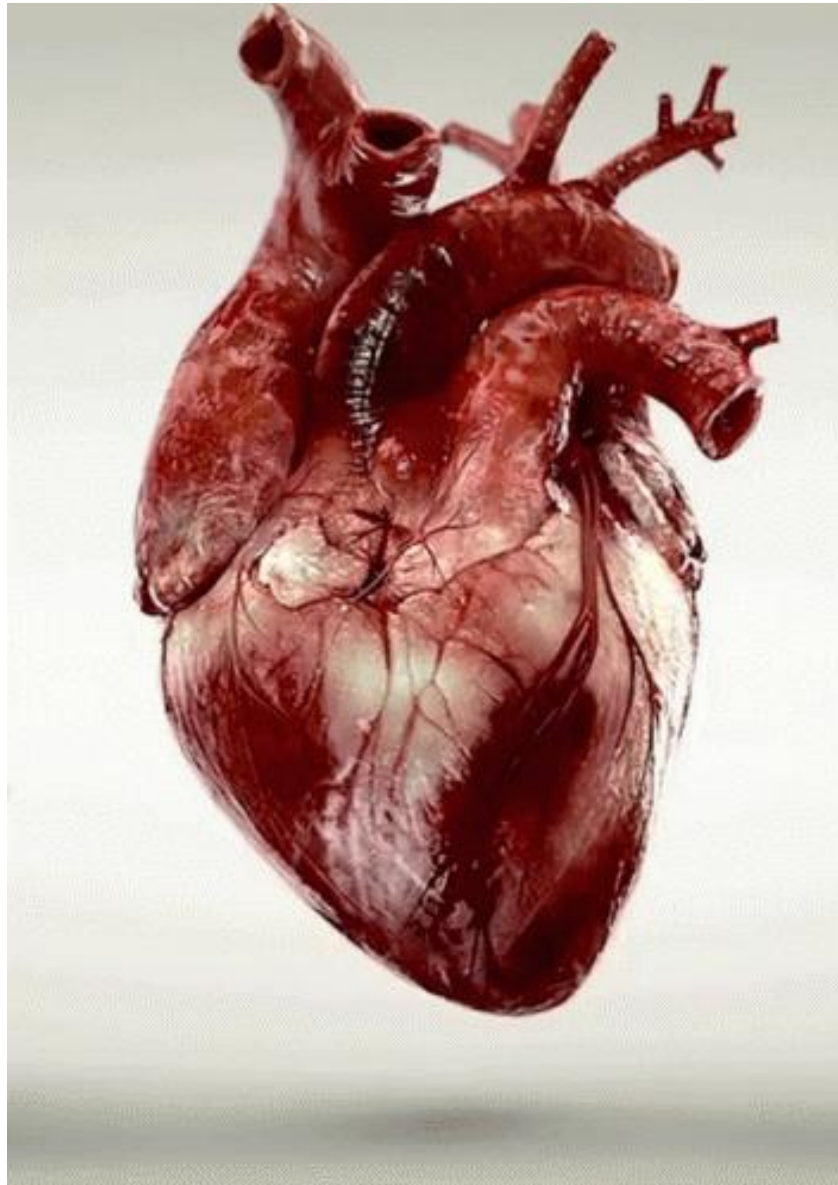
- 목차
 - ECG (Electrocardiogram)
 - PPG (Photoplethysmogram)

ECG – 심장의 전기전도계

● 동방결절 (SA node) → internodal path → 방실결절 (AV node) → His속 (bundle of His) → 심실 (Ventricles)

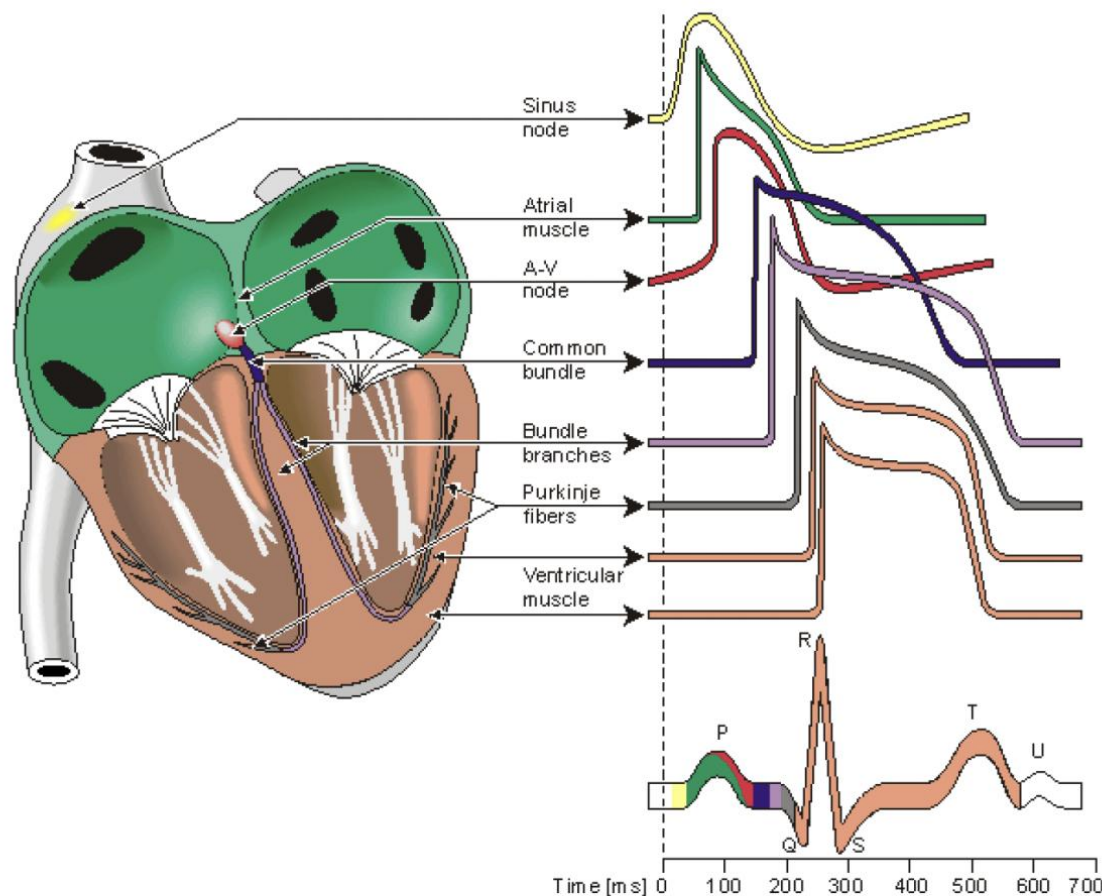


ECG – 심장의 박동



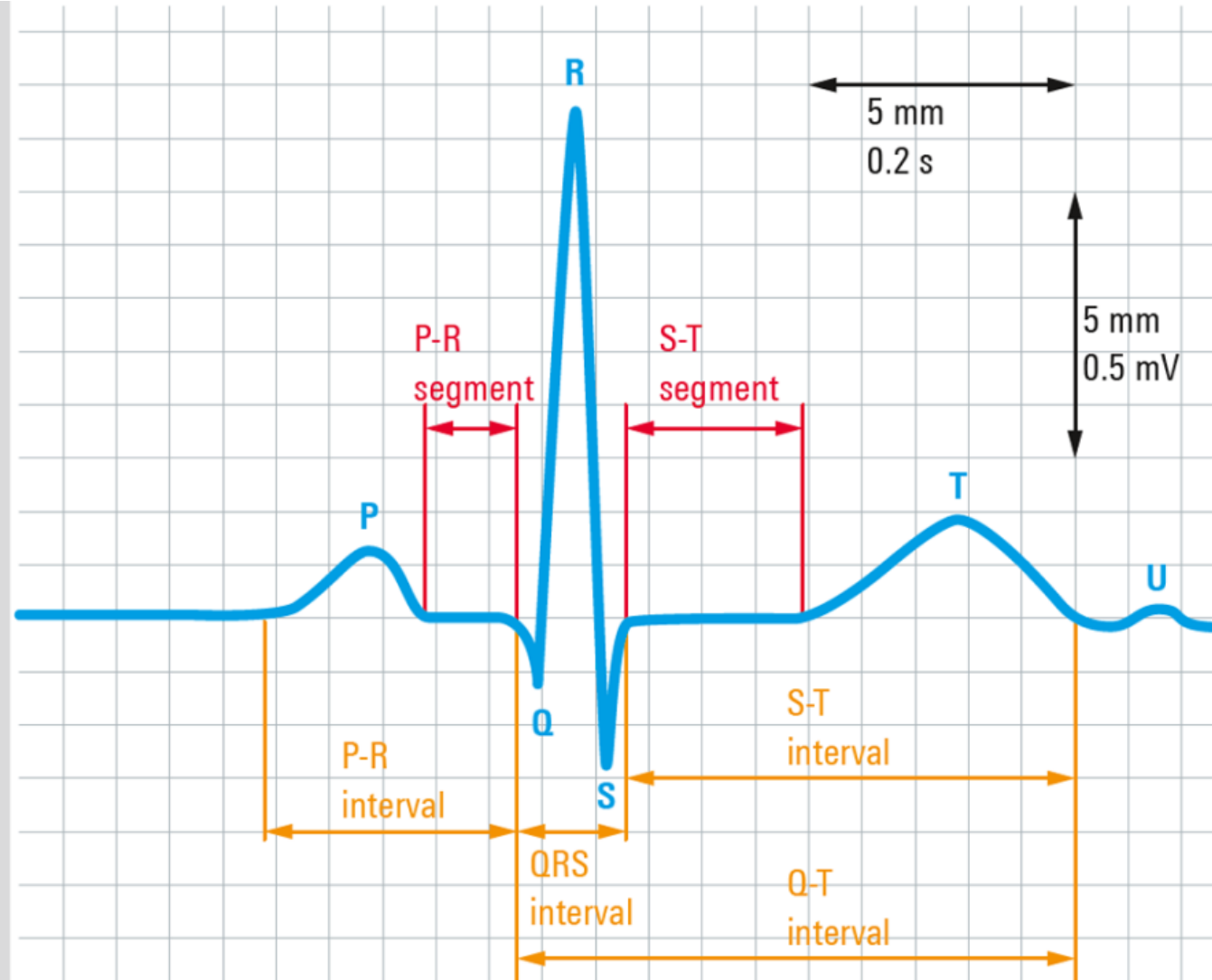
ECG – 심장의 전기 활동과 기록

- 동방결절 (SA node) → internodal path → 방실결절 (AV node) → His속 (bundle of His) → 심실 (Ventricles)

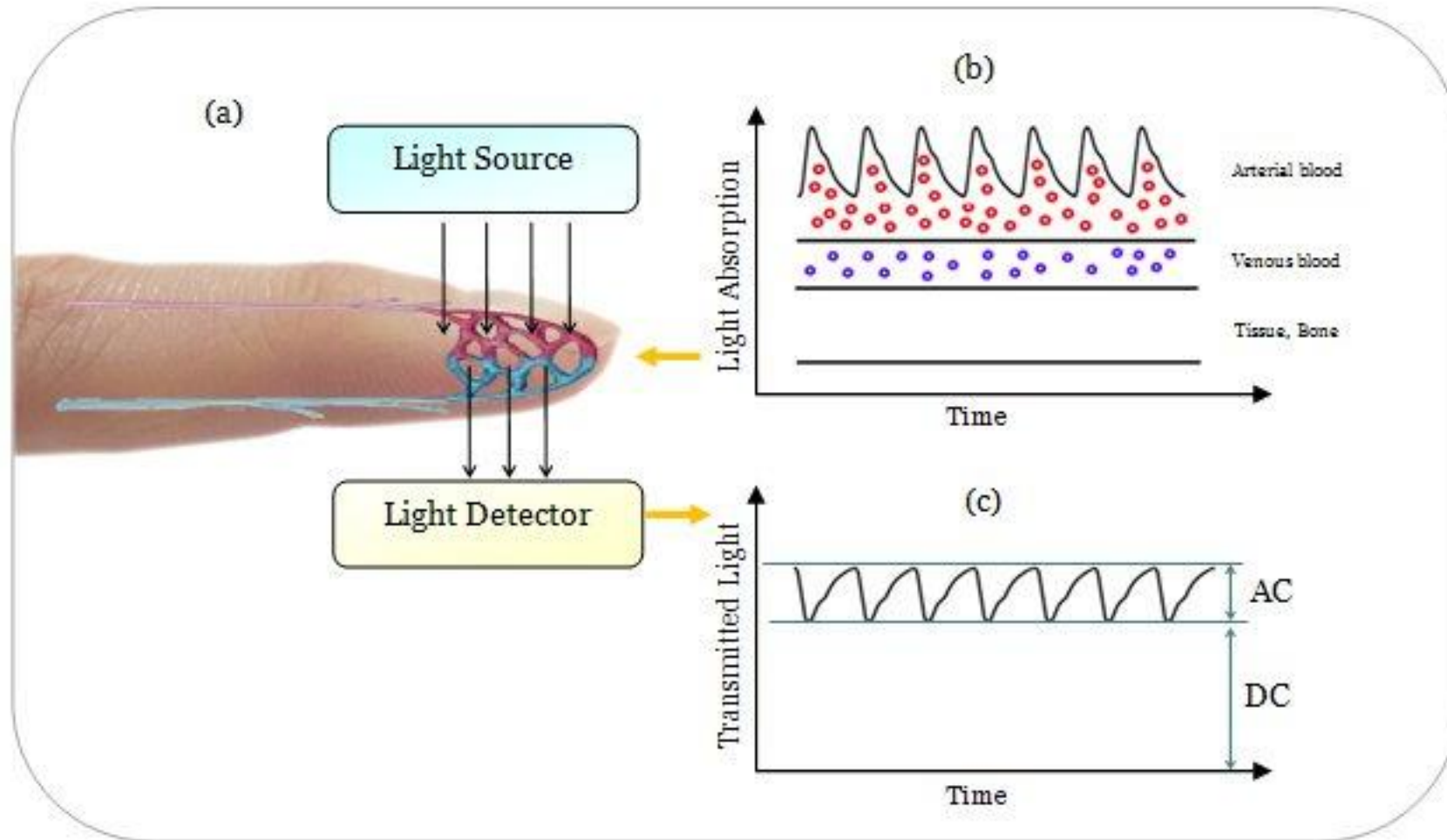


**Electrocardiogram
(ECG/EKG)**

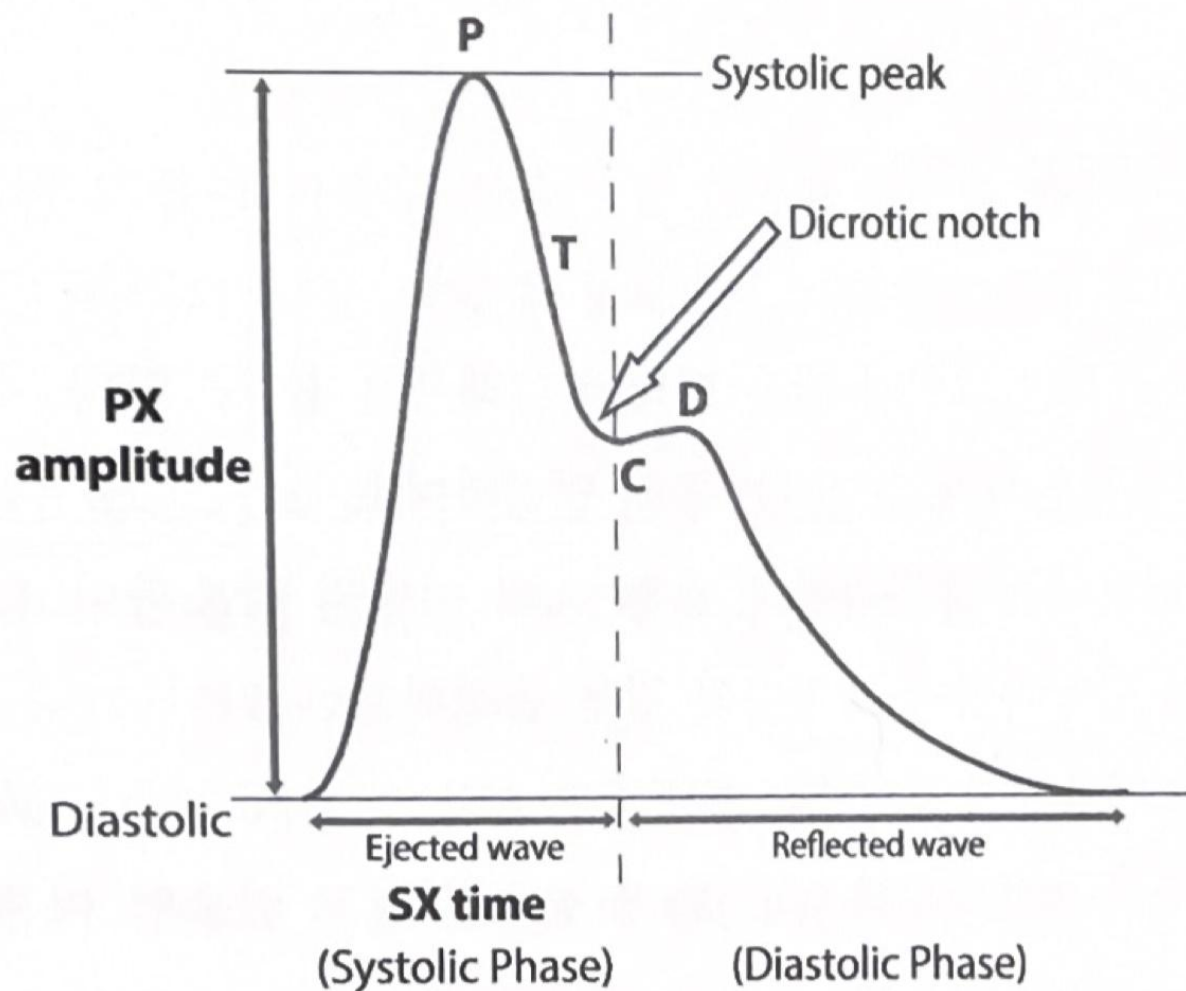
ECG – 구성요소



PPG: Photoplethysmogram

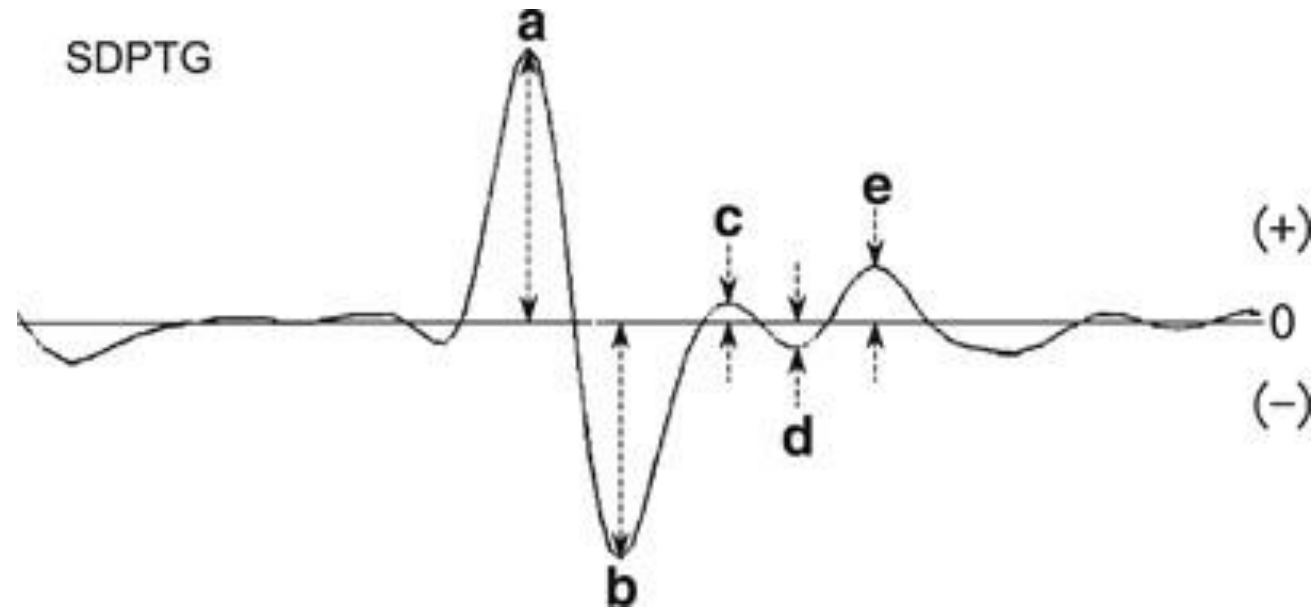


PPG – 구성요소



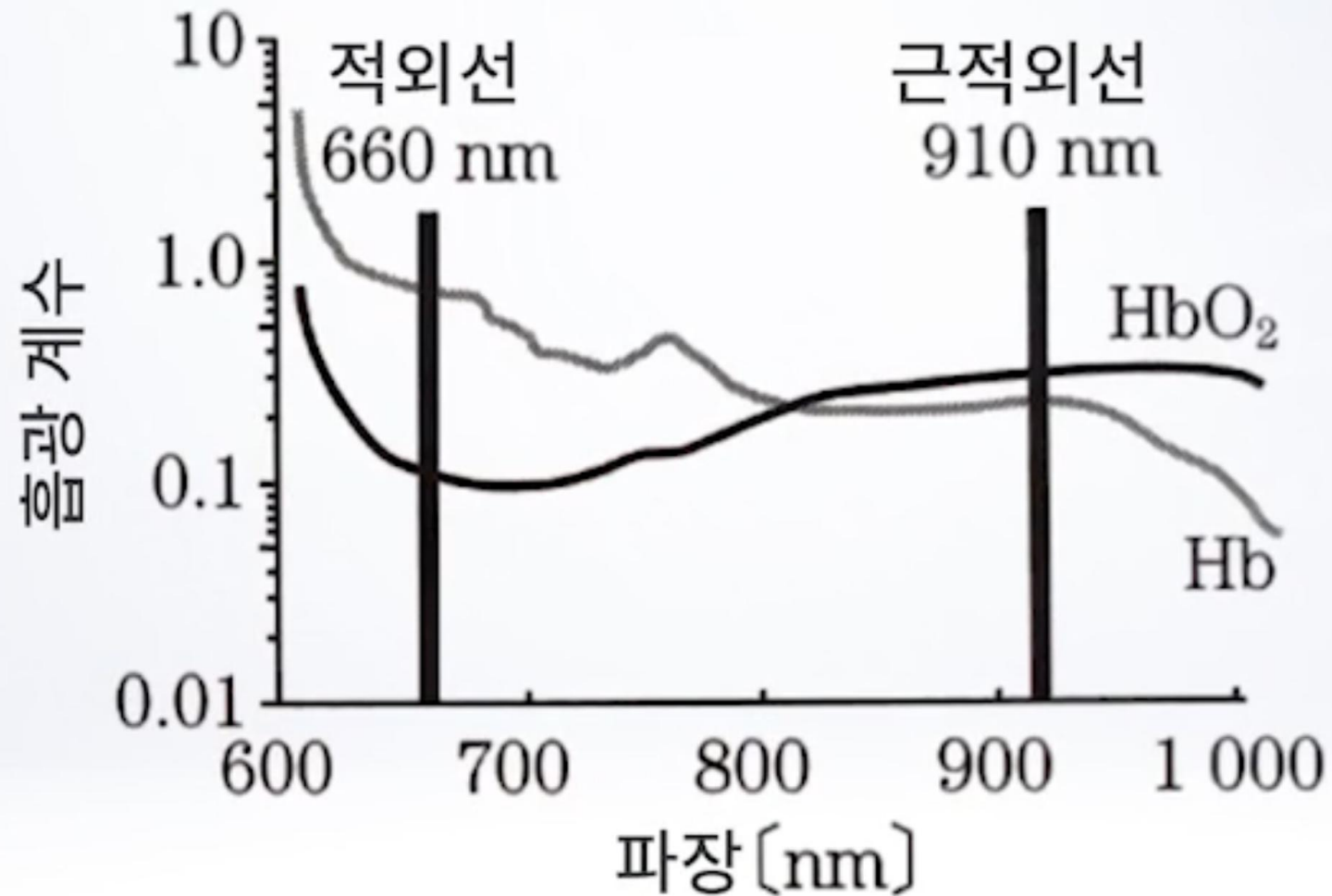
구분	내용
S	대동맥판이 개방되며 좌심실의 혈액이 구출되는 시점
P	혈관내 압력과 용적이 최대에 이르는 시점
T	조랑파. 이전의 맥파성분이 말초에서 반사되어 돌아오는 시점
C	절흔. 대동맥판이 폐쇄되는 시점
D	중복파. 대동맥판 폐쇄 이후 동맥내 압력 및 용적이 근소하게 상승하는 시점
PX ampli.	파고. 혈관탄성이 감소되며 파고가 감소
SX time	승각시간. 혈관상태나 심장의 수축력과 관련있으며 혈관저항이 심수출력보다 상대적으로 증가하면 연장됨
C/P ratio	절흔계수. 절흔 높이에 대한 최대 peak 비. 동맥경화증 등의 진단에 활용함

PPG – Second derivative (SDPPG)

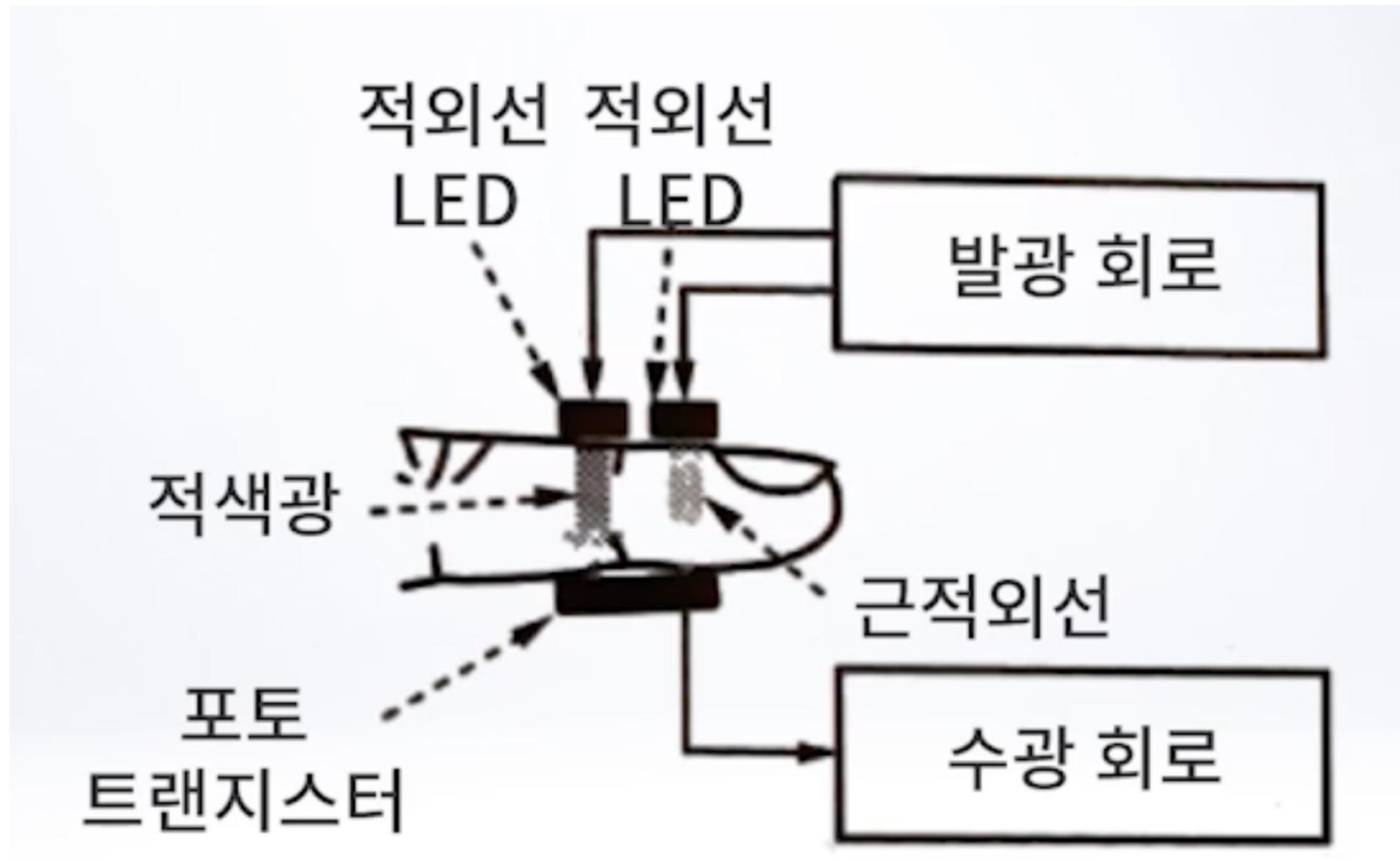


구분	내용
AGI (b-c-d-e/a)	연령지수, 나이가 증가할수록 증가됨. 남성이 여성에 비해 낮음
b/a	혈관의 탄성도. 혈압이 상승하면 증가, 기질적 동맥경화 의심
c/a	대동맥의 탄성도, 연령이 증가함에 따라 감소
d/a	반사파의 증가시 감소 (동맥경화), 총 콜레스테롤이 증가할수록 증가, 나이 미치 몸무게와 상관관계 존재
e/a	나이가 증가함에 따라 감소

PPG – 해모글로빈의 흡광계수

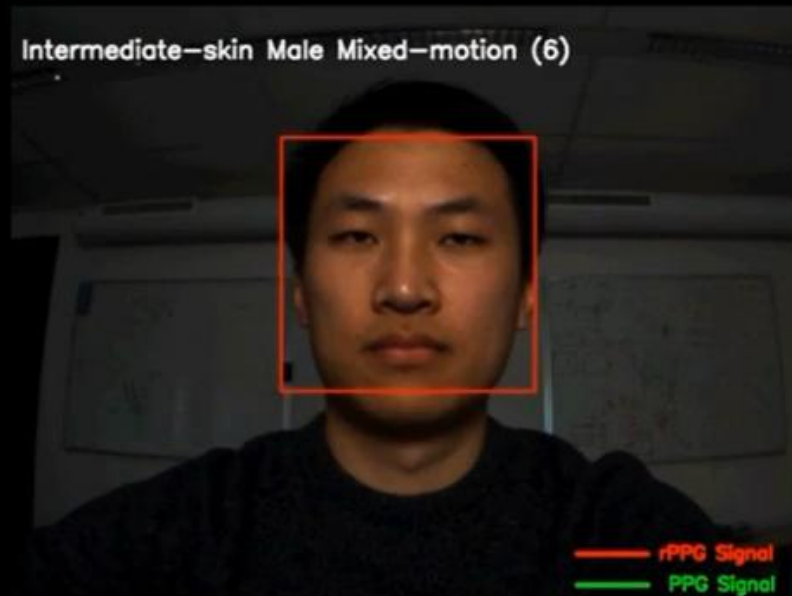


PPG – Pulse Oximeter using PPG

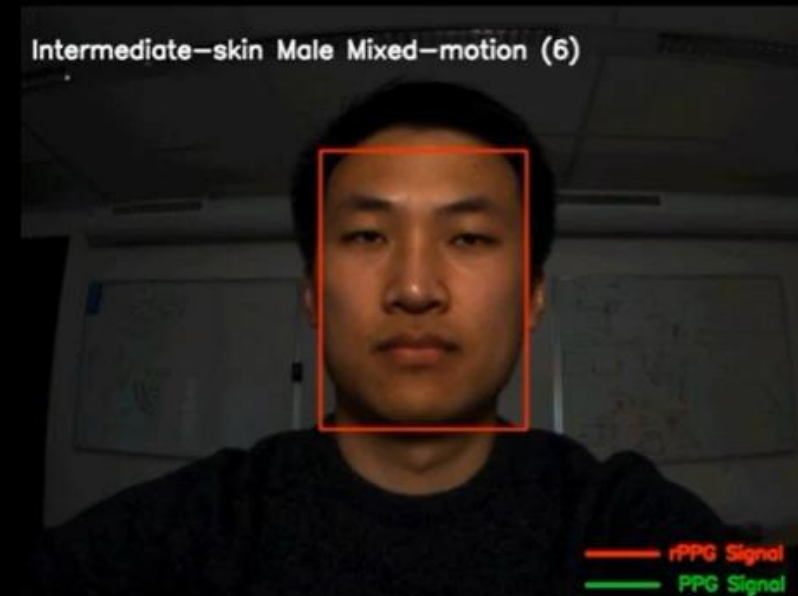


Contactless camera-based PPG

Existing method (2013)



Proposed method (2014)



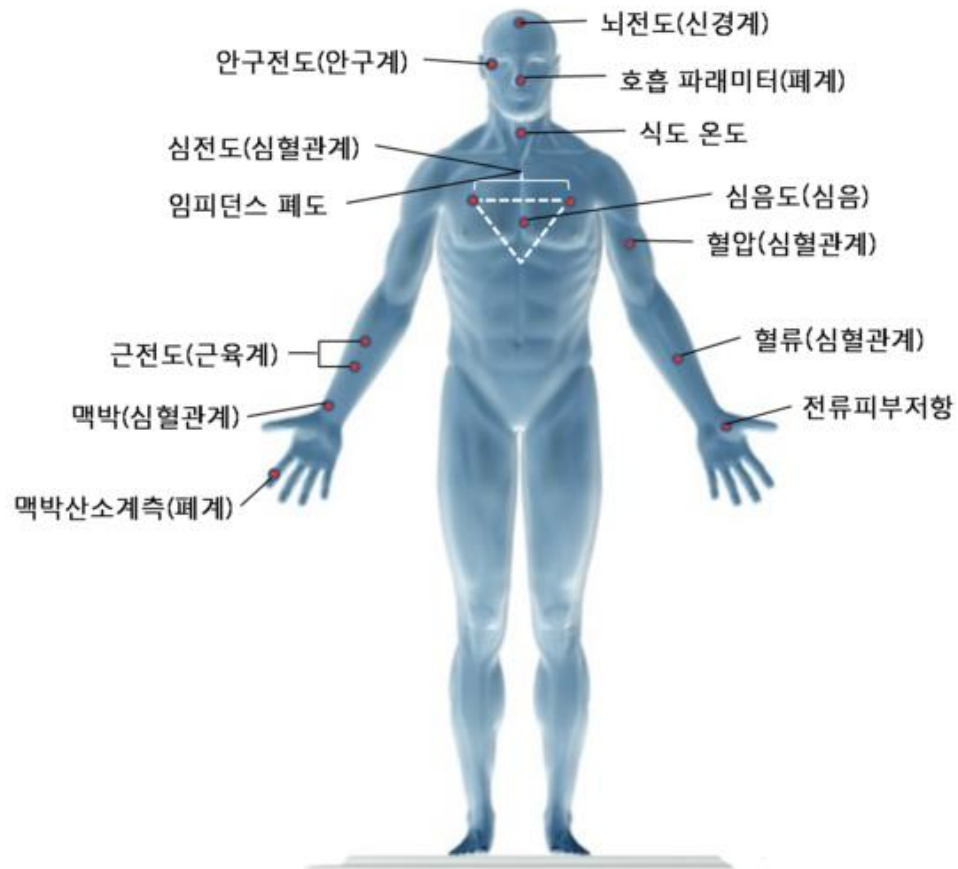
Red pulse-signal is camera-based rPPG signal
Green pulse-signal is contact-based PPG signal (Ground-truth)

Contactless camera-based PPG



- 목차
 - 생체신호 측정
 - 신호 처리 과정
 - 신호와 주파수
 - 신호의 종류와 ADC
 - 신호처리의 필요성

Measurement of physiological signals



Processing of physiological signal

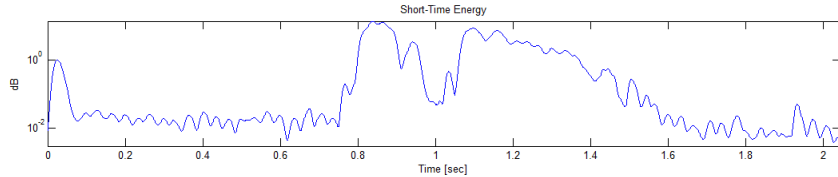
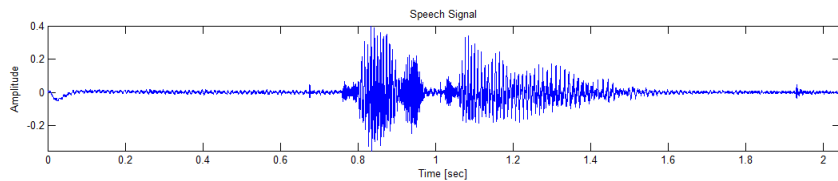
- 1. 센서를 통해 측정된 신호를 전기 신호로 전환
- 2. 전기신호를 디지털 신호로 변환
- 3. 원하는 생체신호의 주파수 대역으로 필터링
- 4. 신호를 증폭하여 표출
 - 생체신호는 $\mu V \sim mV$ 로 크기가 매우 작음



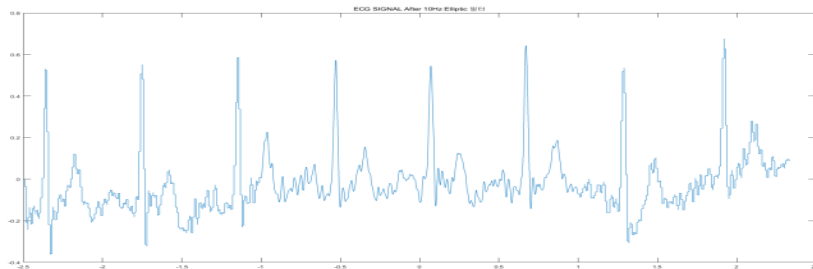
신호와 주파수

● Hz (주파수)

- 1초당 진동 횟수 : 1/cycle, Hz
- 실제 자연상태의 모든 신호는 다양한 대역의 주파수가 섞여 있음

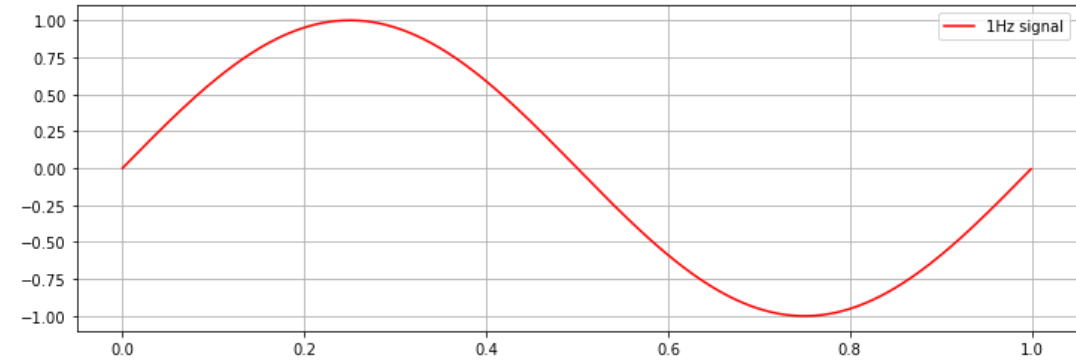


음성 신호의 예시

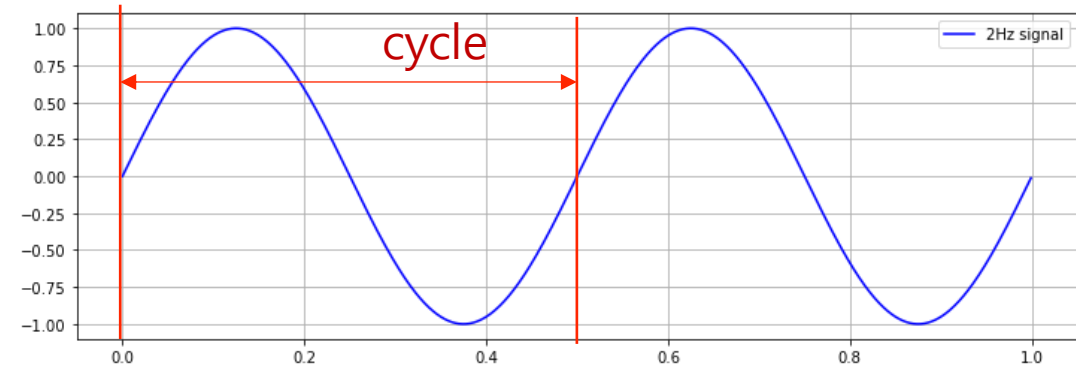


심전도 신호의 예시

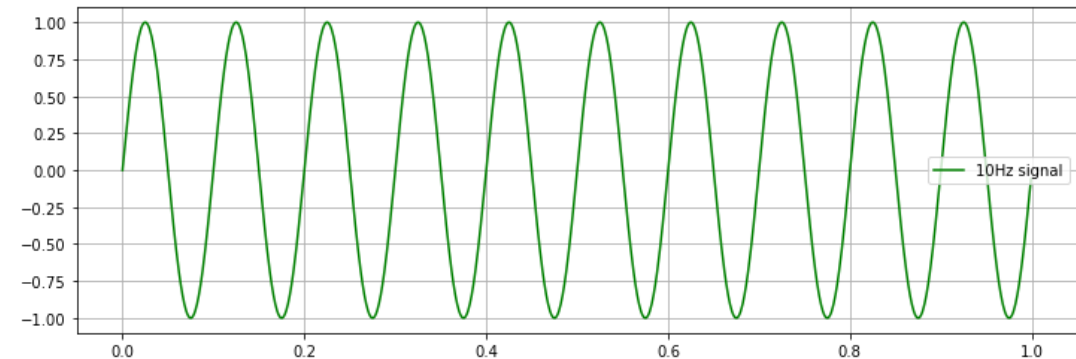
1Hz



2Hz



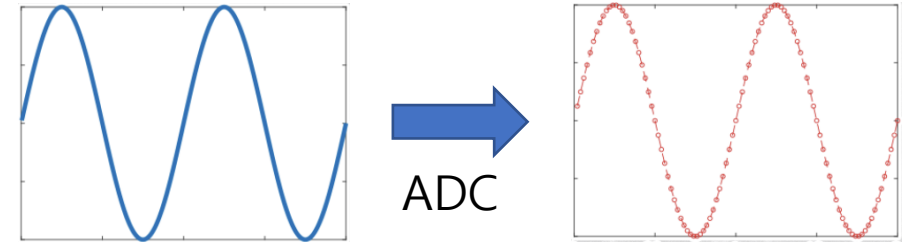
10Hz



신호의 종류와 ADC

● 연속신호 (Continuous signal)

- **Analog signal** 이라고도 부름
- 외부적인 원인에 의해서 연속적으로 변화하는 것들을 물리량으로 나타낸 것
- 일반적으로 존재하는 자연상태의 정보는 연속적인 신호

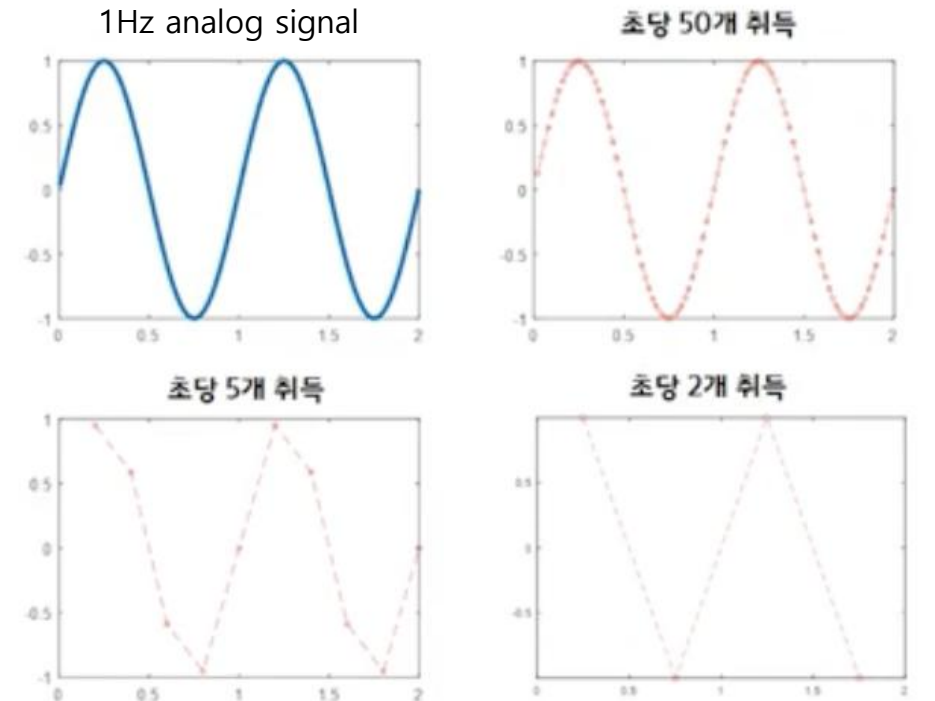


● 이산신호 (Discrete signal)

- 컴퓨터를 이용한 데이터 처리를 위해 연속적인 데이터를 이산신호로 변환해야 함
- 스위치 등을 이용해 on/off 해가며 연속된 신호를 비연속적으로 취득함

● 아날로그-디지털 변환 (Analog to Digital Conversion; ADC)

- Sampling rate: 얼마 간격으로 데이터를 취득 (샘플링) 할 것인가?
- 원 신호의 주파수 특성을 고려하여 취득 필요
- 원 신호의 최대 주파수의 2배 이상으로 취득해야 함 (Nyquist Sampling Theorem)



신호처리의 필요성

- 측정된 생체신호는 우리의 관심 신호 뿐 아니라 다른 파형의 신호도 함께 포함되어 있음

- 대표적인 노이즈: 전원 노이즈

- *국내 전원 규격: 60Hz

- 따라서 우리가 원하는 신호만 추출하기 위해 필터링이 필요함



Filtering – Convolution (합성곱)

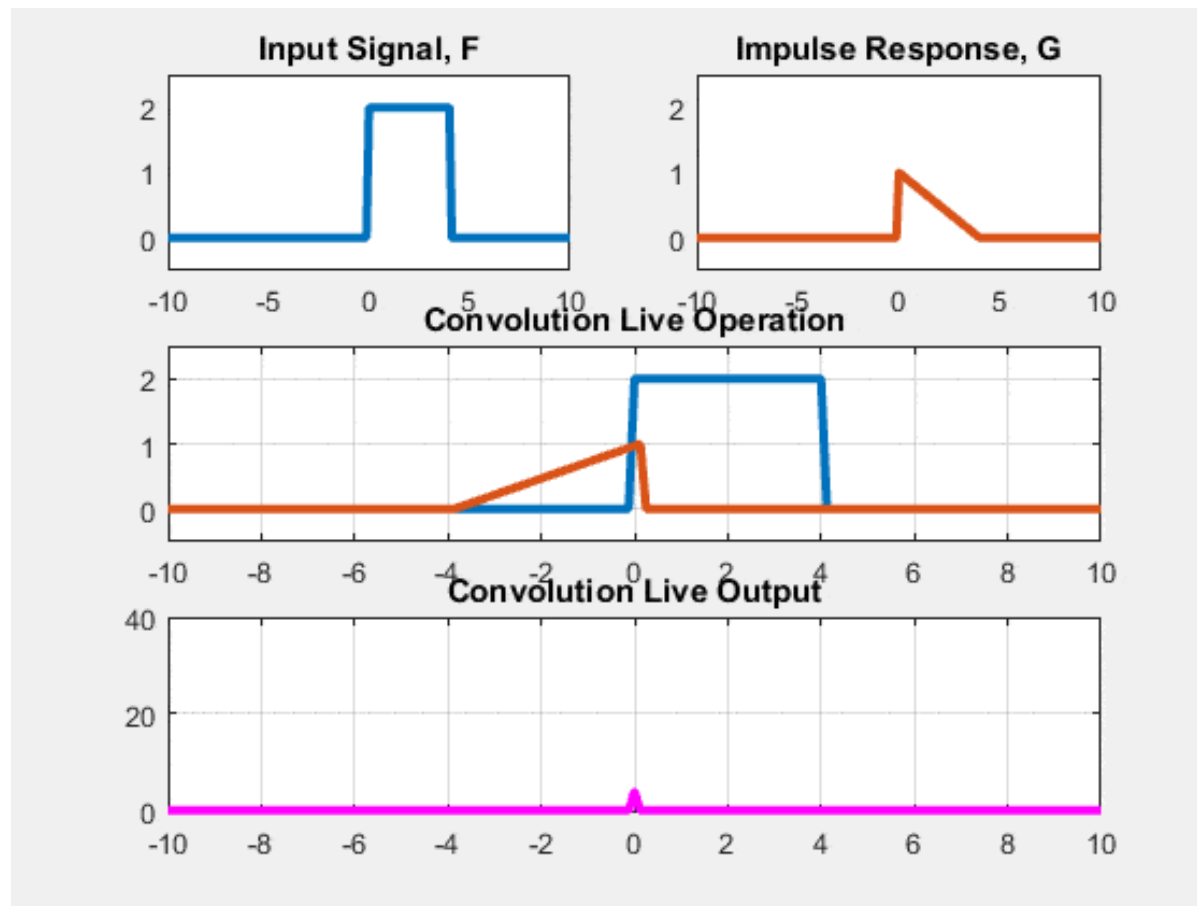
- 두 함수를 곱하고 적분하여 (더하여) 새로운 함수를 구하는 수학 연산자

- 전자분야의 “Convolution”에서는 반응 함수를 반전시켜 연산을 수행함
(Convolution Neural Network 안에서의 “convolution” 연산은 엄밀히는 “cross-correlation”임)

- 두 함수 f, g 가 있을 때, convolution ($f * g$)는,

- $$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

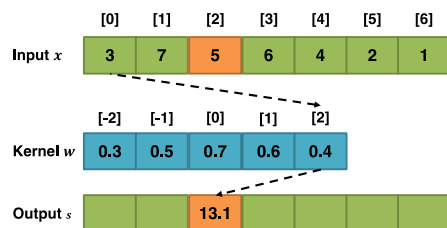
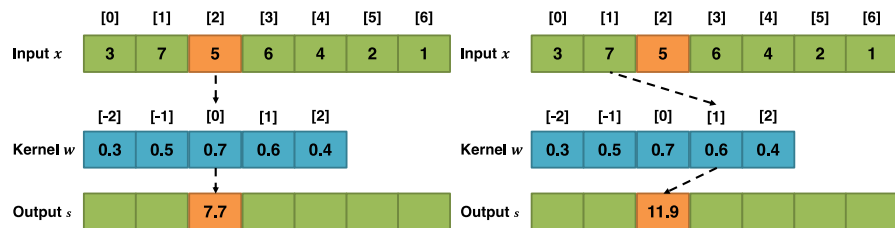
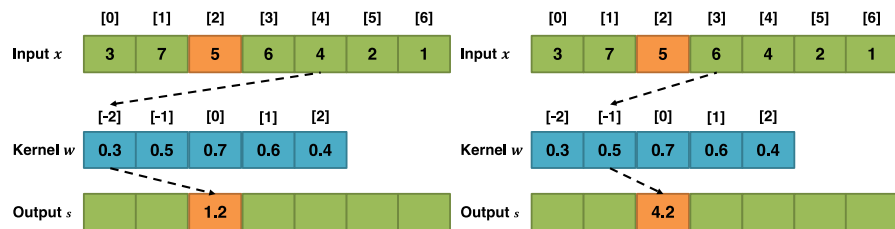
- Response function 을 잘 설계하면
필터링을 수행할 수 있음



참고) Convolution vs Cross-correlation

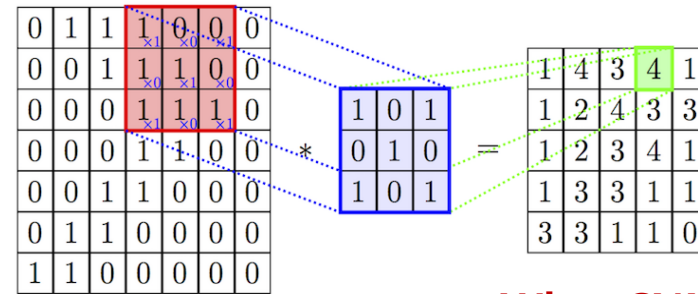
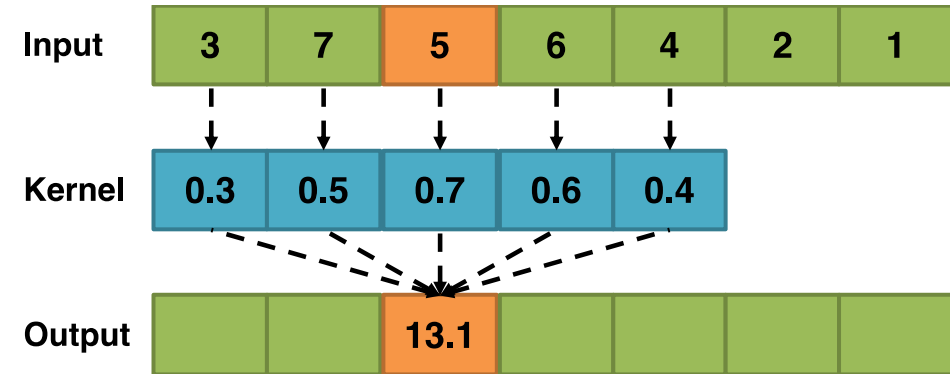
Convolution

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n).$$



Cross-correlation

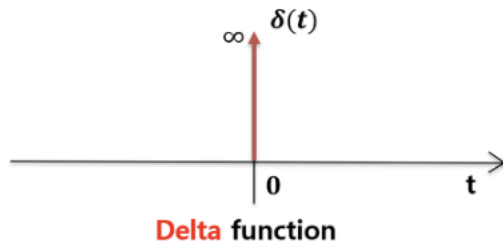
$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$



What CNN actually borrowed!

Impulse function

- Delta function: 0인 순간에서만 값을 무한대로 가지고 나머지 영역에서 0의 값을 갖는 함수
 - 이 경우의 값을 1로 간주하였을 때, **Unit impulse function**이 됨

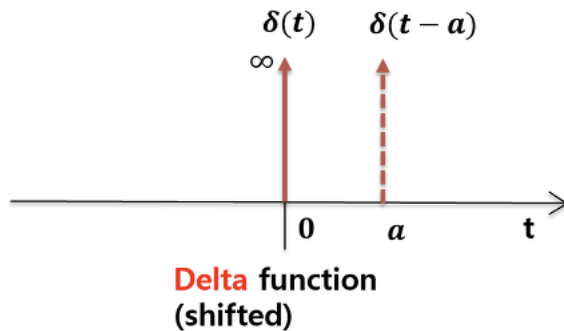


Delta function :

$$\delta(t) = \begin{cases} +\infty & t=0 \\ 0 & t \neq 0 \end{cases}$$

$$\int_{-\infty}^{\infty} \delta(t) dt = 1$$

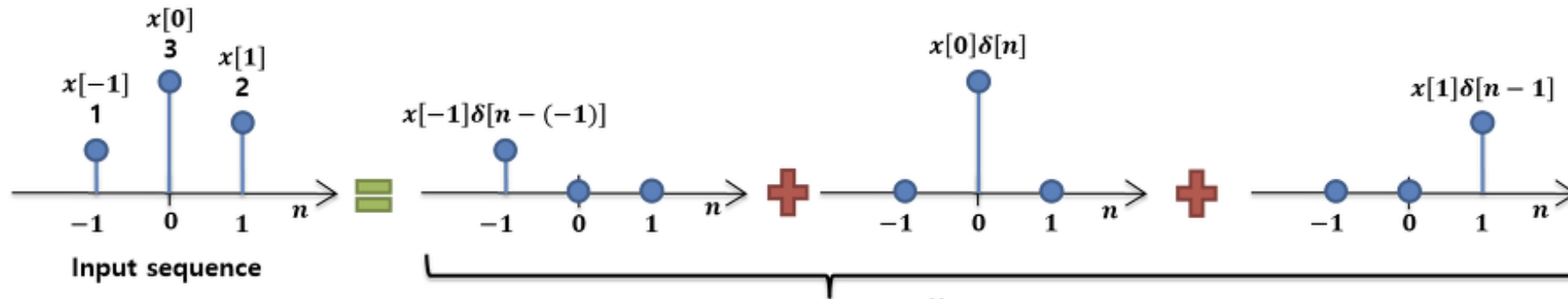
- Unit impulse function을 Response function으로 사용하여 convolution을 수행하면, $t=0$ 인 구간의 신호 값만 추출할 수 있음
- 또한, 이를 shift 해가며 우리가 원하는 구간의 값을 추출할 수 있음



Shifted Delta function :

$$\delta(t-a) = \begin{cases} +\infty & t=a \\ 0 & t \neq a \end{cases}$$

Signal decomposition

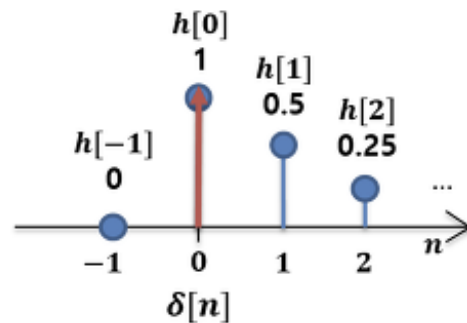


$$x[n] = \sum_{k=-\infty}^{\infty} x[k]\delta[n - k]$$

Impulse Response of LTI System

Ex) Impulse response

$$h[n] = \left(\frac{1}{2}\right)^n u[n]$$

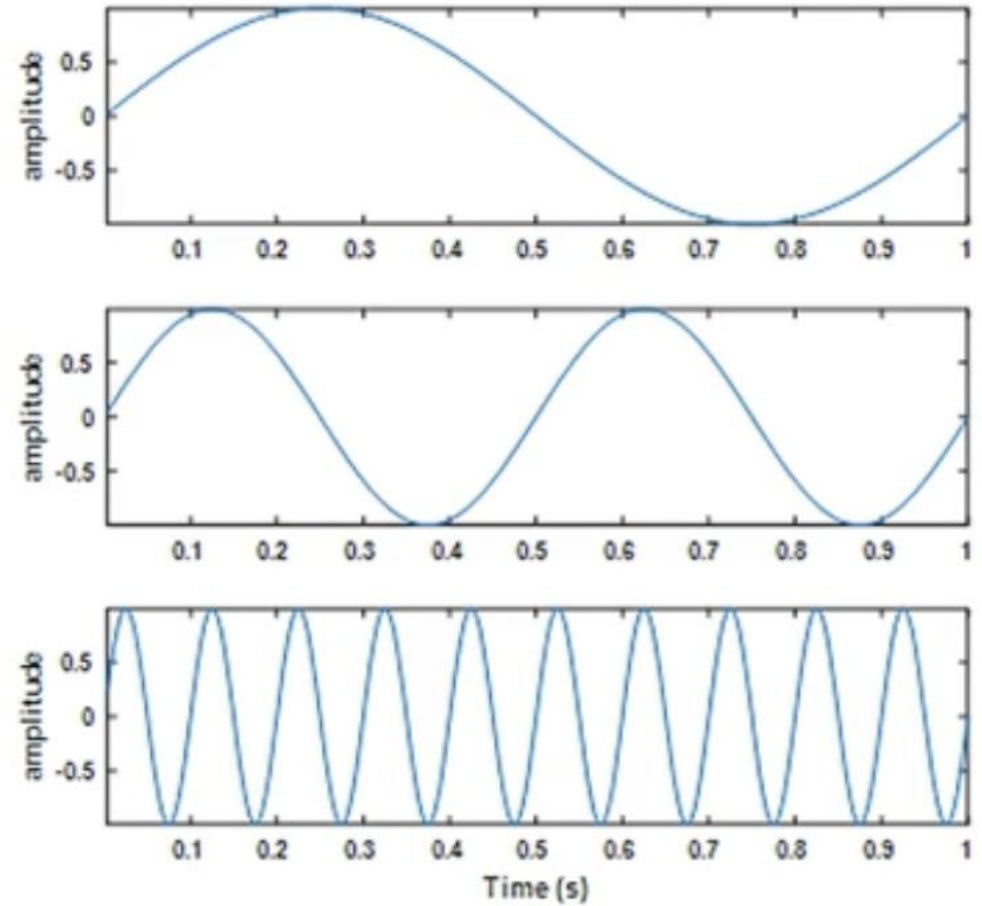
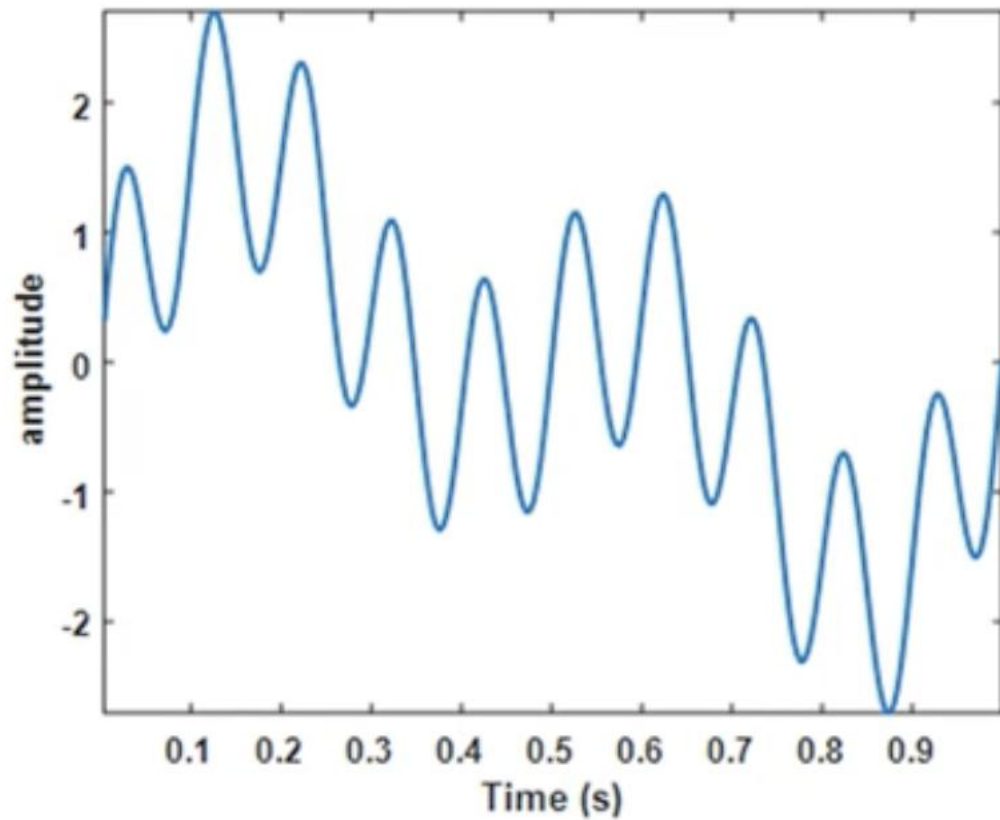


Convolution

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n - k]$$

주파수 분석

- 한 신호에 섞여있는 신호를 분석하여 불필요한 신호를 제거함



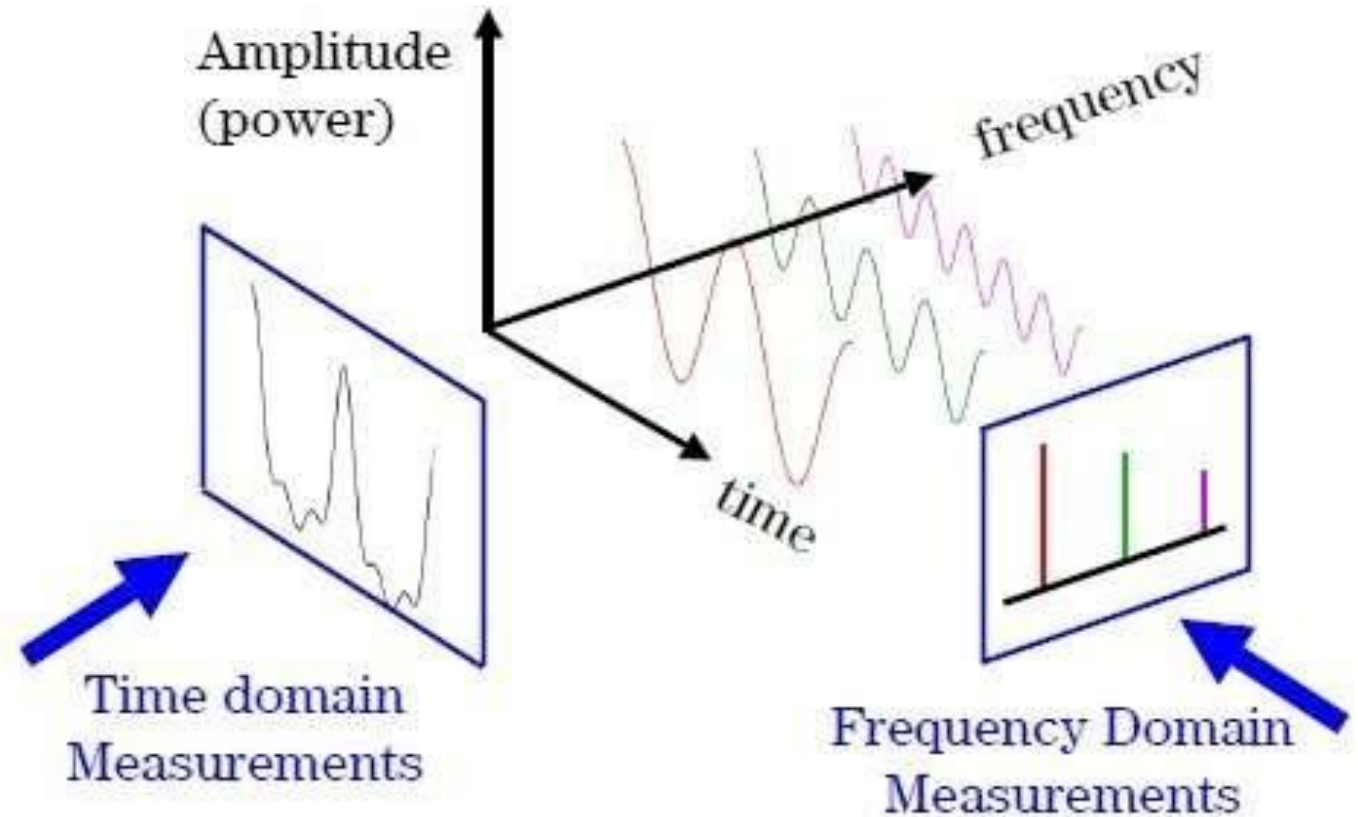
Fourier transform (푸리에 변환)

● 푸리에 급수 (Fourier series)

- 모든 신호는 주파수가 서로 다른 $\sin$, $\cos$ 함수의 합으로 표현할 수 있다.

● 푸리에 변환 (Fourier Transform)

- 시간이나 공간에 대한 함수를 시간 또는 공간 주파수 성분으로 분해하는 변환
- 활용 예:
심전도: 0.1Hz ~ 200Hz
근전도: 25Hz ~ 5000Hz



Fourier transform (푸리에 변환)

● 시간 - 주파수 관계

- 시간 영역에서의 필터링은 합성곱
- 주파수 영역에서 필터링은 곱

$$x[n] * y[n] \leftrightarrow N \times X[k]Y[k]$$

시간영역

주파수영역

$$z(t) = x(t) * y(t) = \int_0^T x(\tau) y(t - \tau) d\tau$$

$$Z[k] = \frac{1}{T} \int_0^T z(t) e^{-jk\omega_0 t} dt$$

$$= \frac{1}{T} \int_0^T \int_0^T x(\tau) y(t - \tau) e^{-jk\omega_0 t} dt d\tau$$

$$= \frac{1}{T} \int_0^T \int_0^T x(\tau) y(t - \tau) e^{-jk\omega_0(t-\tau)} e^{-jk\omega_0\tau} dt d\tau$$

$$= \frac{1}{T} \int_0^T x(\tau) e^{-jk\omega_0\tau} \left[\int_0^T y(t - \tau) e^{-jk\omega_0(t-\tau)} dt \right] d\tau$$

$$= \frac{1}{T} \int_0^T x(\tau) e^{-jk\omega_0\tau} T \left[\frac{1}{T} \int_0^T y(t - \tau) e^{-jk\omega_0(t-\tau)} dt \right] d\tau$$

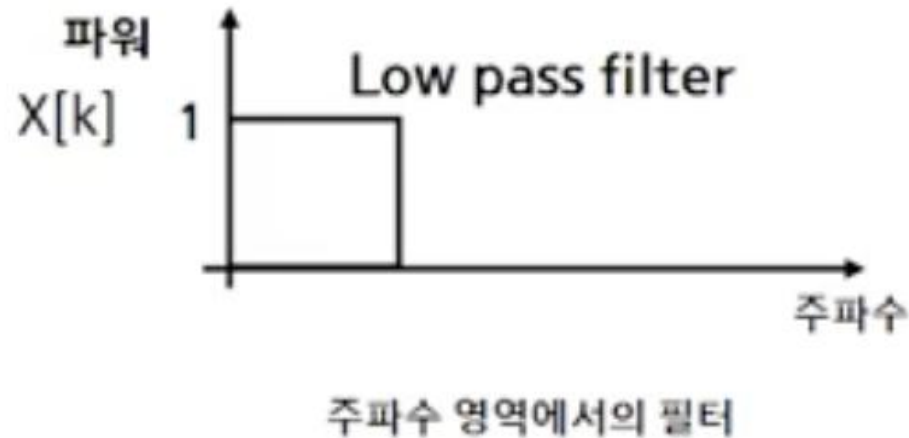
$$= \frac{1}{T} \int_0^T x(\tau) e^{-jk\omega_0\tau} T Y[k] d\tau$$

$$= T Y[k] \frac{1}{T} \int_0^T x(\tau) e^{-jk\omega_0\tau} d\tau = T Y[k] X[k]$$

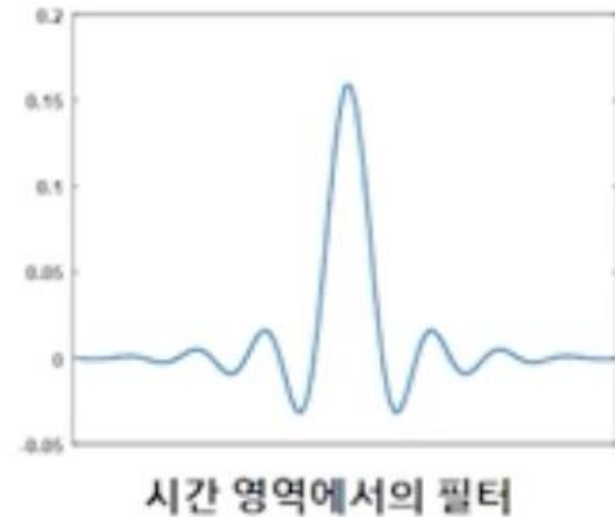
Fourier transform (푸리에 변환)

● 시간 - 주파수 관계

- 저주파의 성분을 꺼내오는 필터를 구현하고자 할 때,
- 주파수 영역에서 바라본 필터는 박스 형태 (step function)를 가짐
- 이를 시간 영역으로 변환하면 **sinc function** 형태의 필터를 가짐



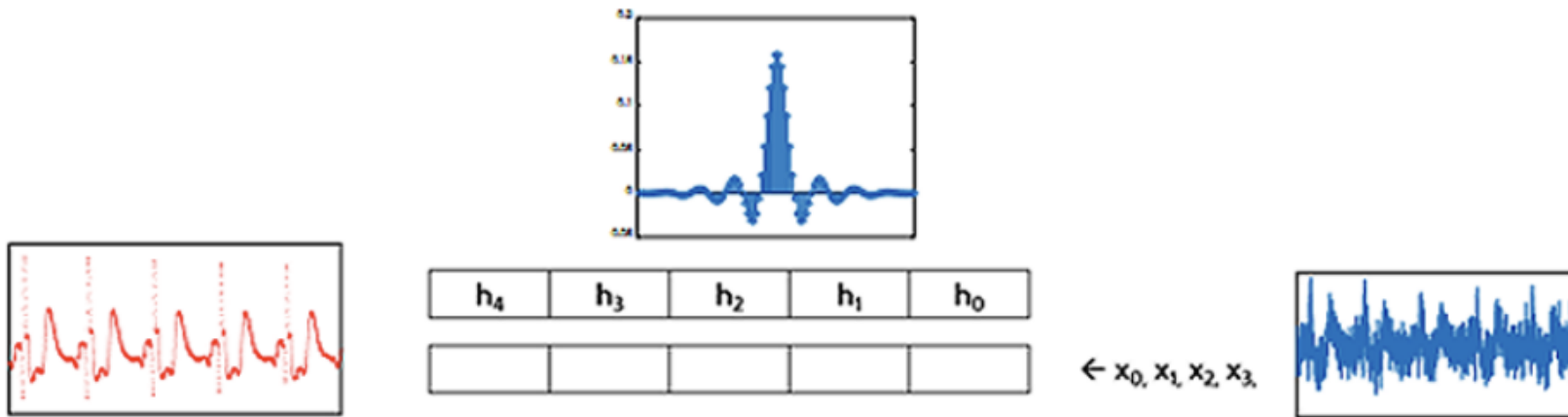
푸리에변환



Finite Impulse Response (FIR)

● 시간 영역에서의 필터링

- 추출하고자 하는 주파 대역에 맞는 sinc function을 바탕으로, 원본 신호에 convolution을 수행함
- 해당 주파 대역에 해당되는 신호만을 추출할 수 있음



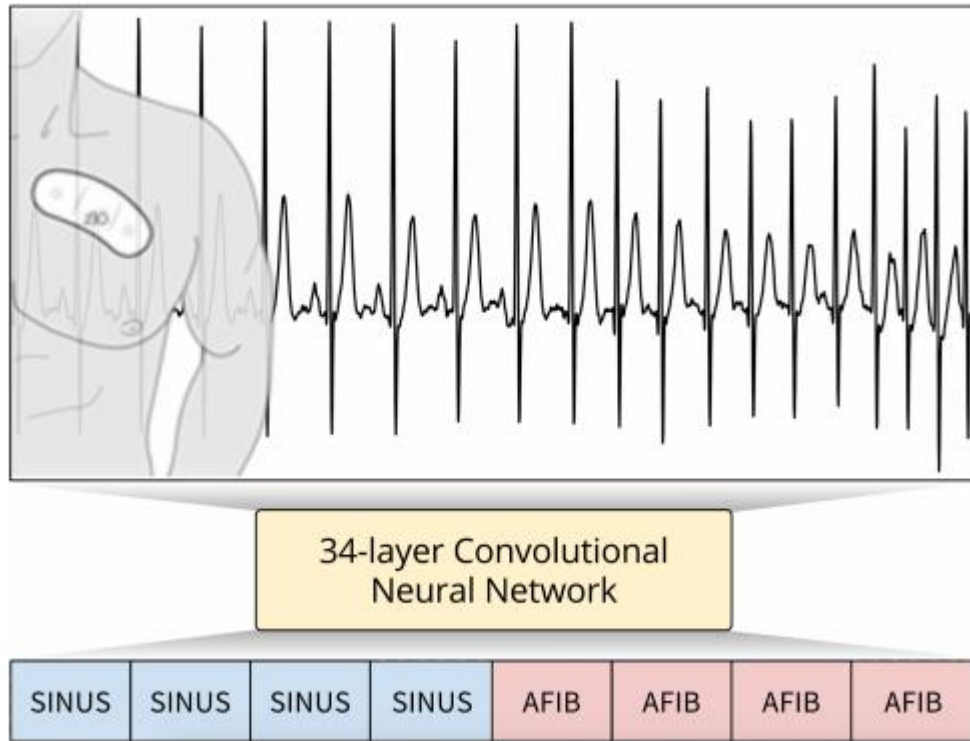
$$y_0 = h_0 x_0$$

$$y_1 = h_0 x_1 + h_1 x_0$$

$$y_2 = h_0 x_2 + h_1 x_1 + h_2 x_0$$

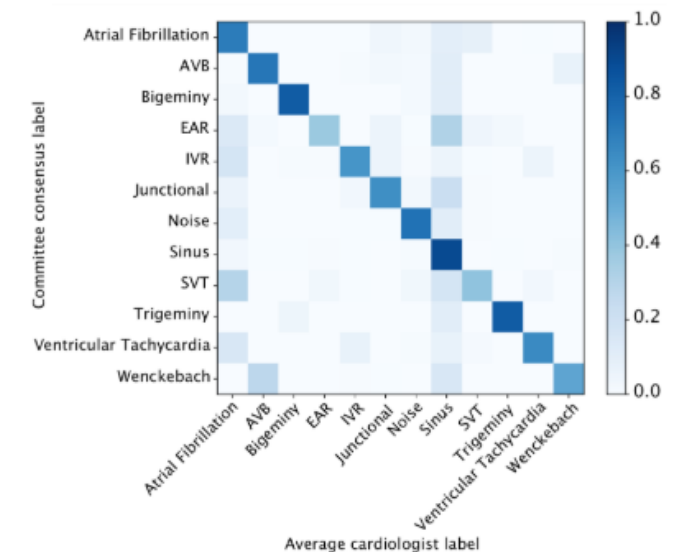
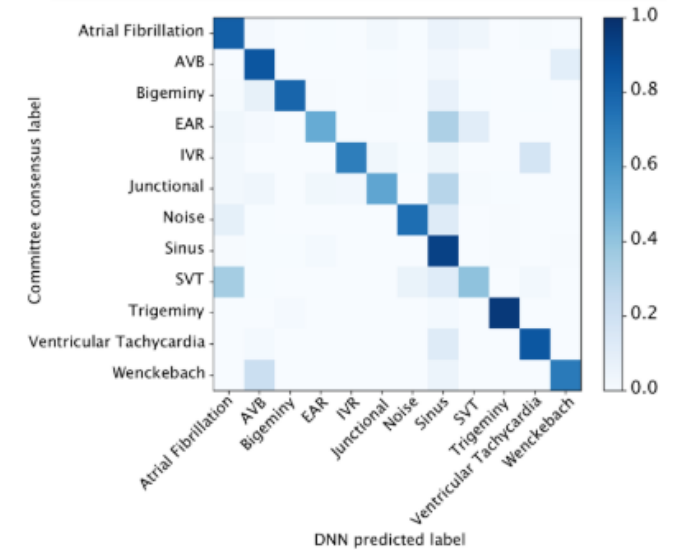
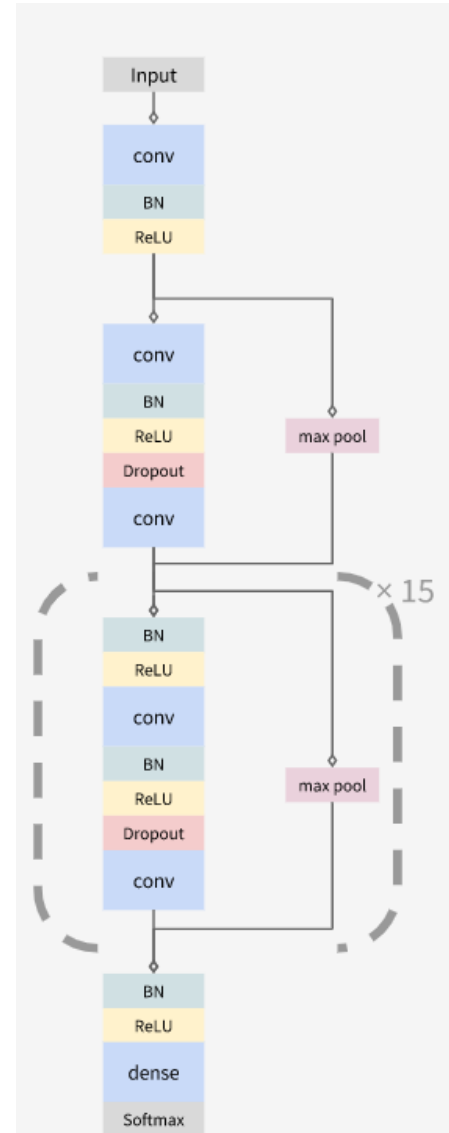
...

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$
$$= \sum_{k=-\infty}^{\infty} h[k] \cdot x[n-k]$$

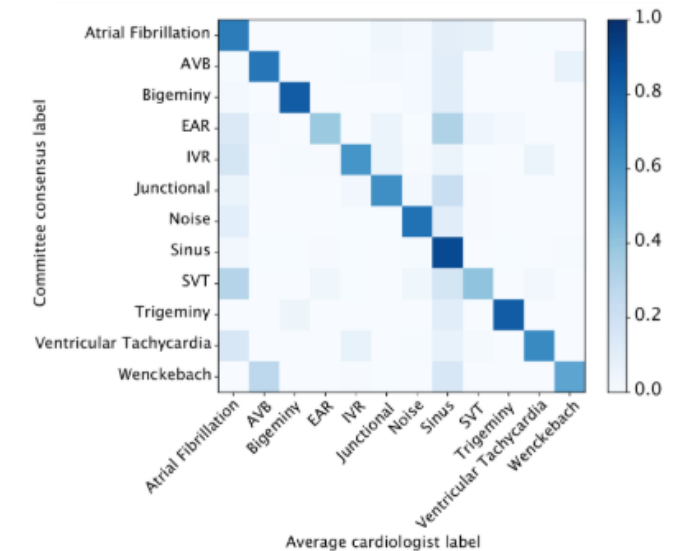
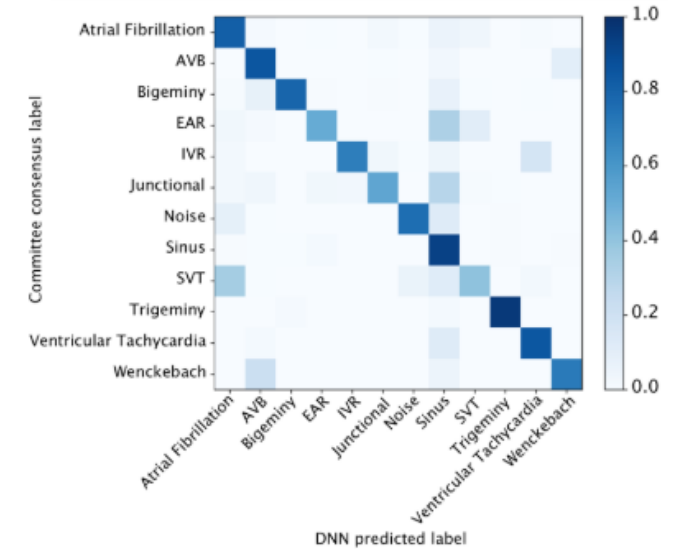
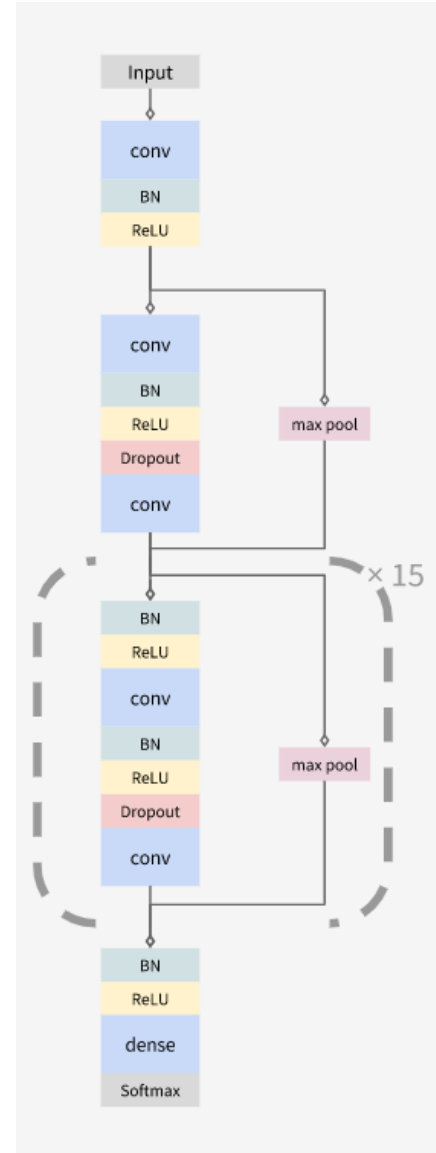
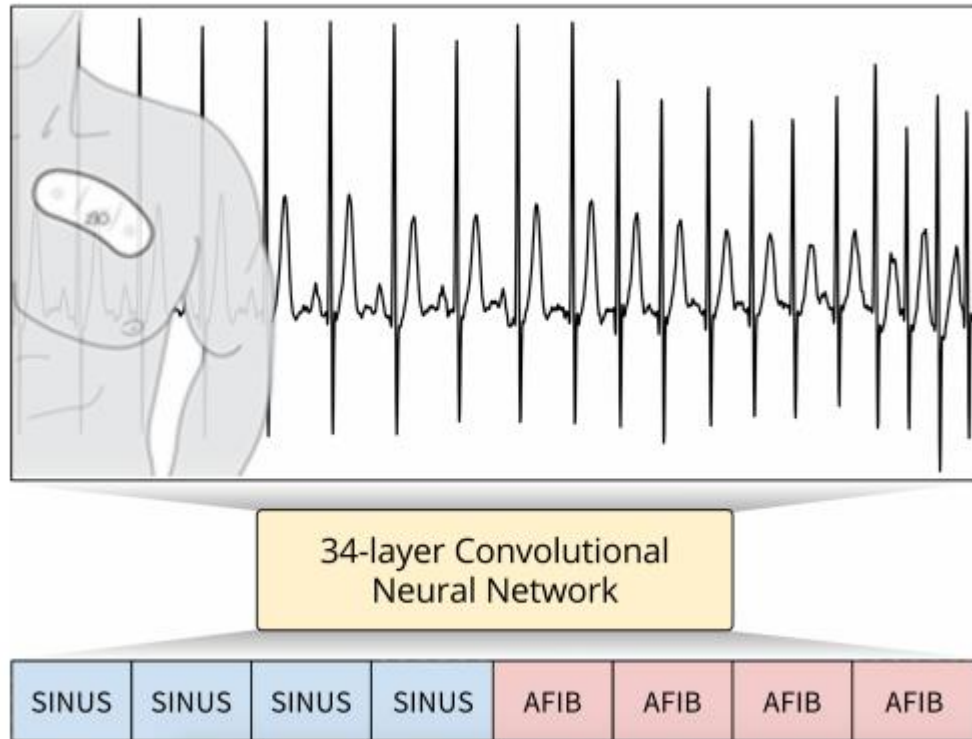


Source: Stanford ML Group

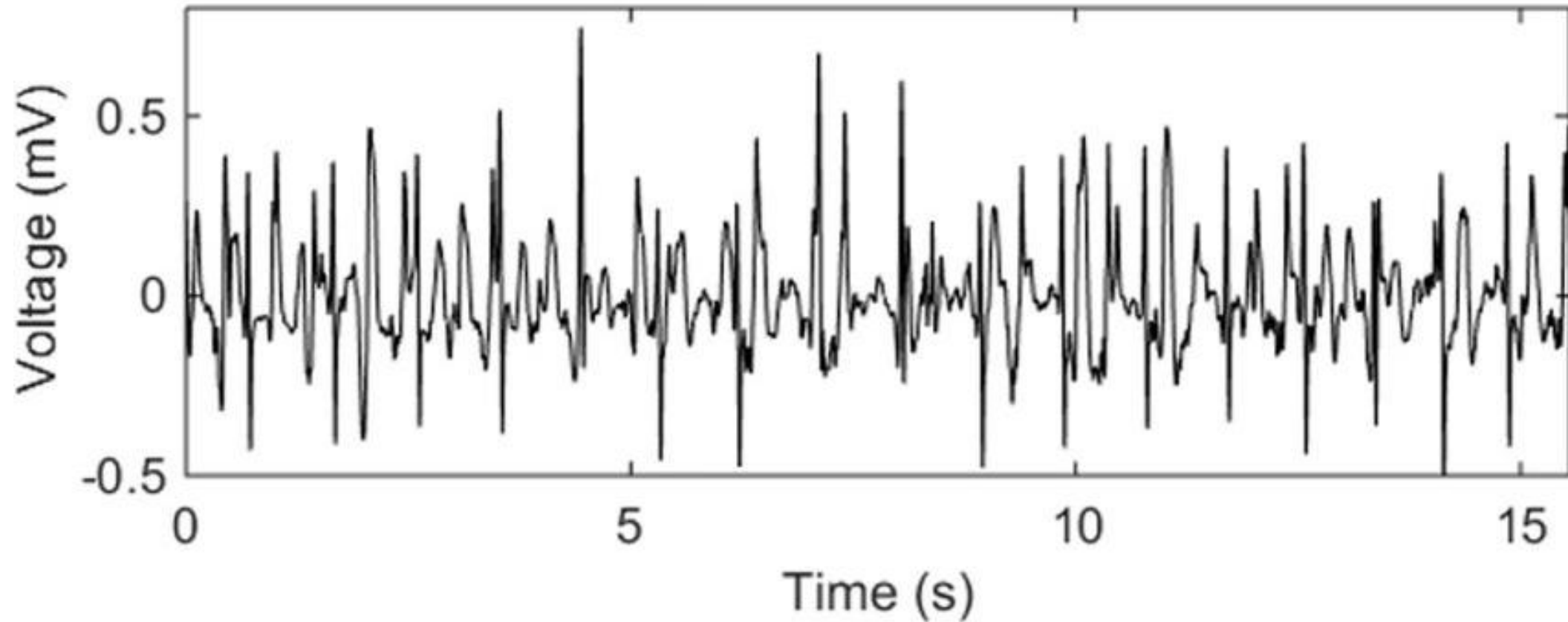
Hannun, A.Y., Rajpurkar, P., Haghpanahi, M. *et al.* Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network. *Nat Med* **25**, 65–69 (2019). <https://doi.org/10.1038/s41591-018-0268-3>



What we need is ...

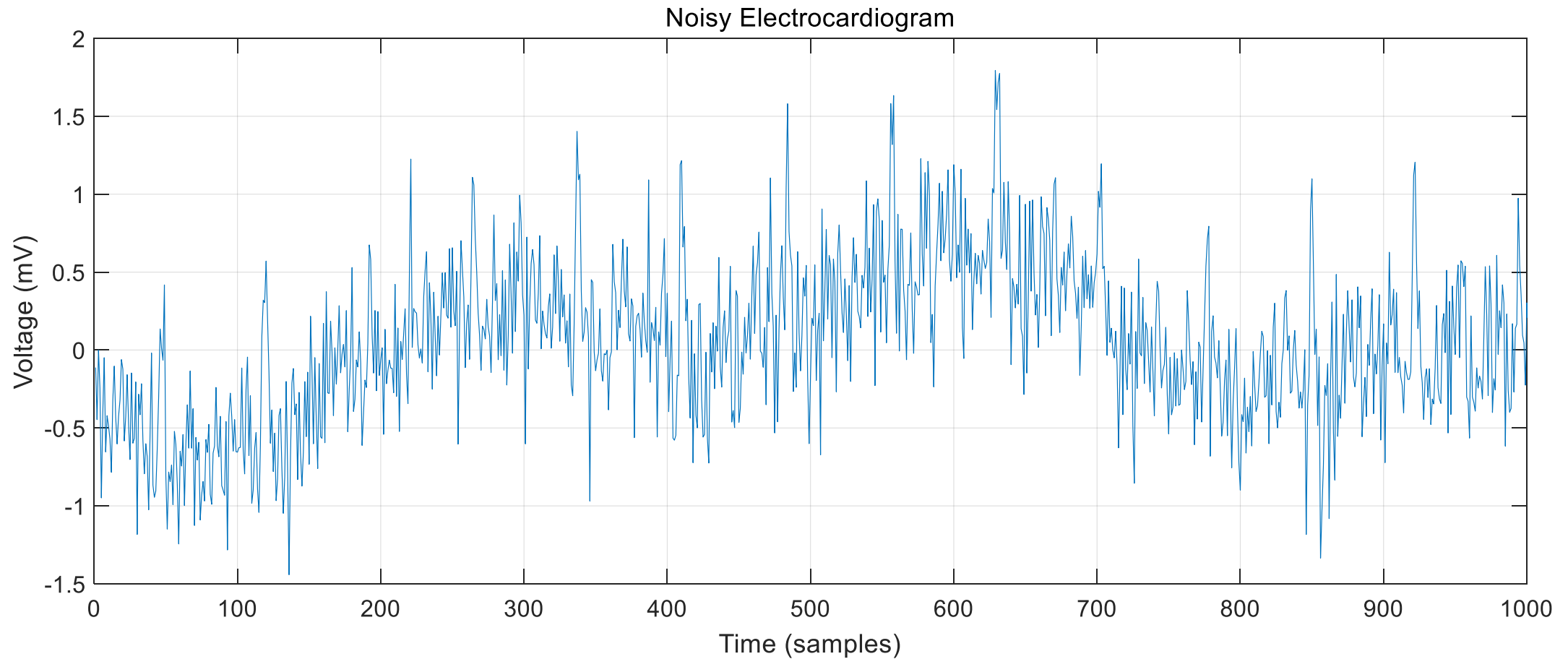


What we have is ...



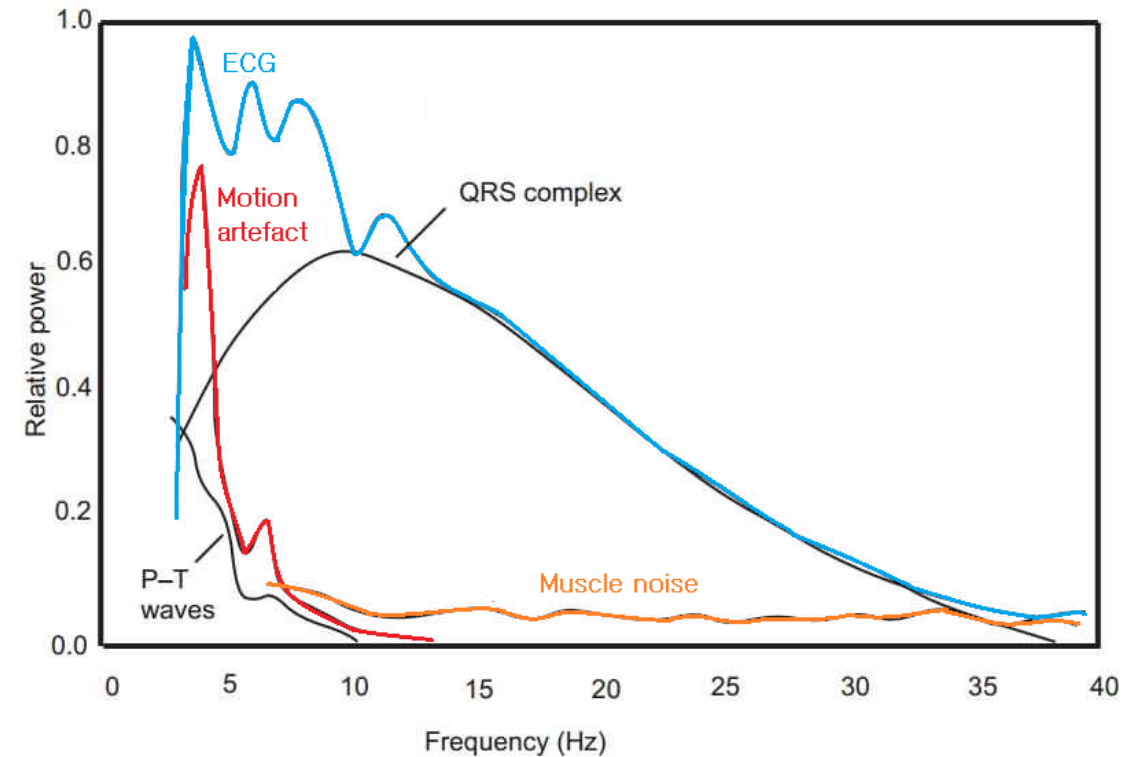
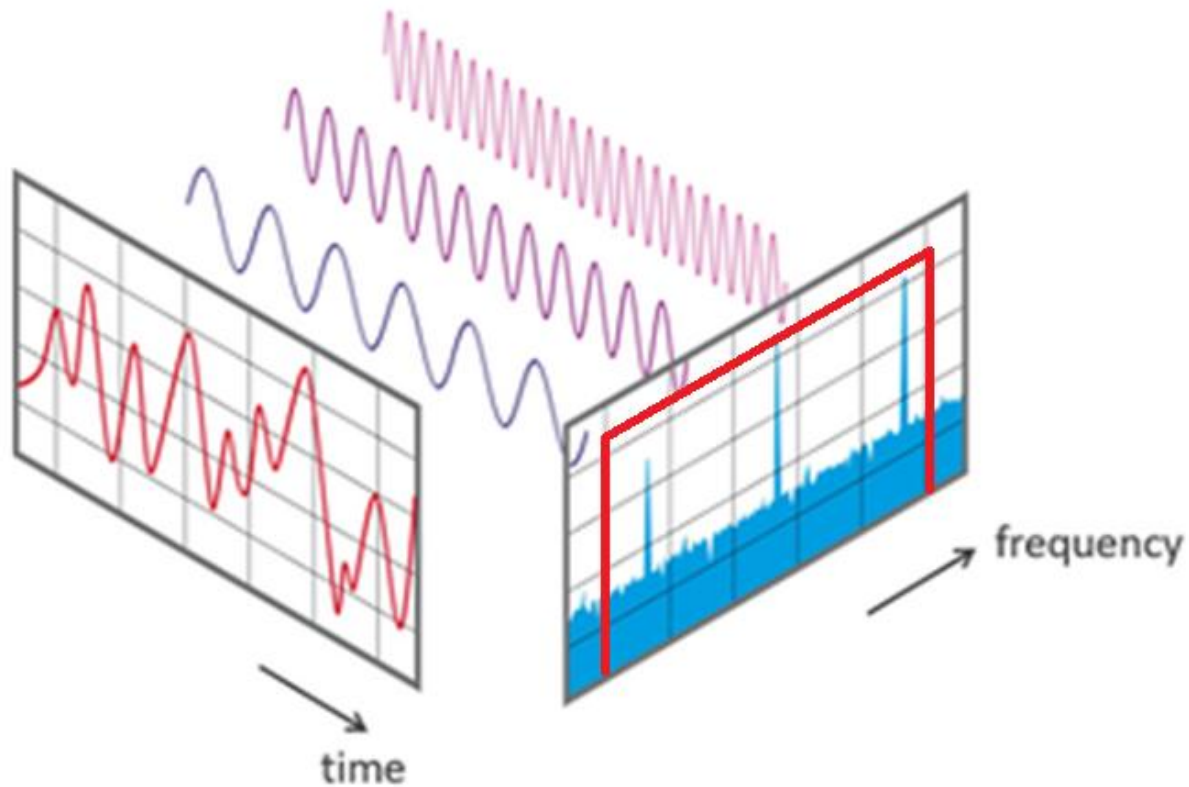
Especially true for recordings with movement or in uncontrolled setting!

What we have is ...



Especially true for recordings with movement or in uncontrolled setting!

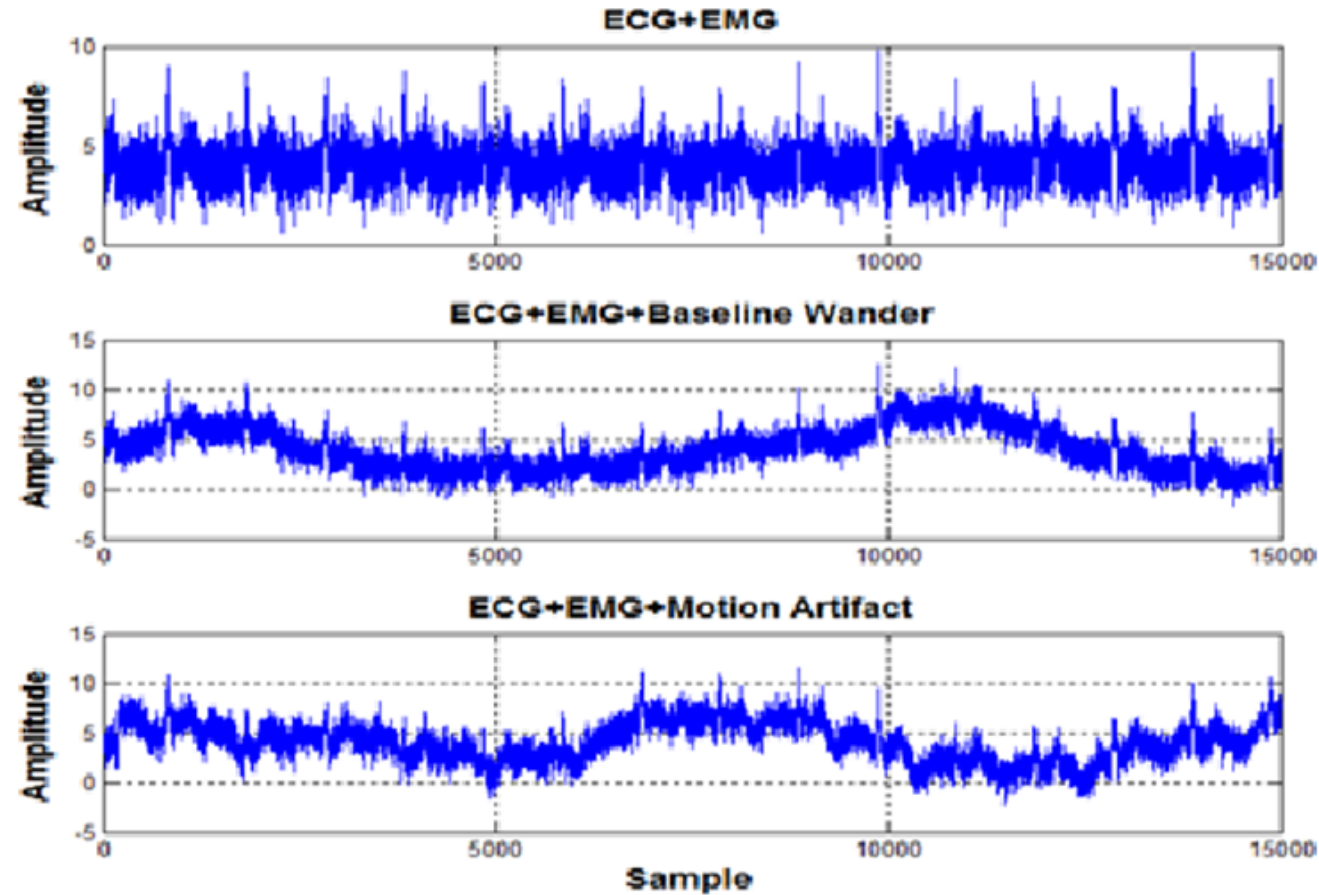
Physiological signal filtering



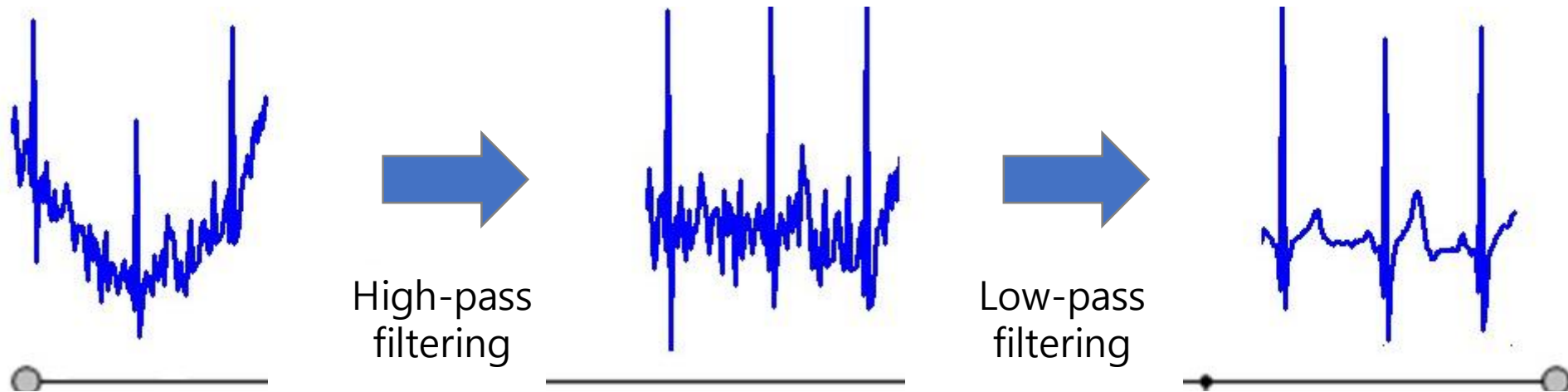
Limitations

- Some Noises (especially motion artefacts) tend to be non-stationary
- Noises can lie within the bandpass of the frequency filter

Physiological signal filtering



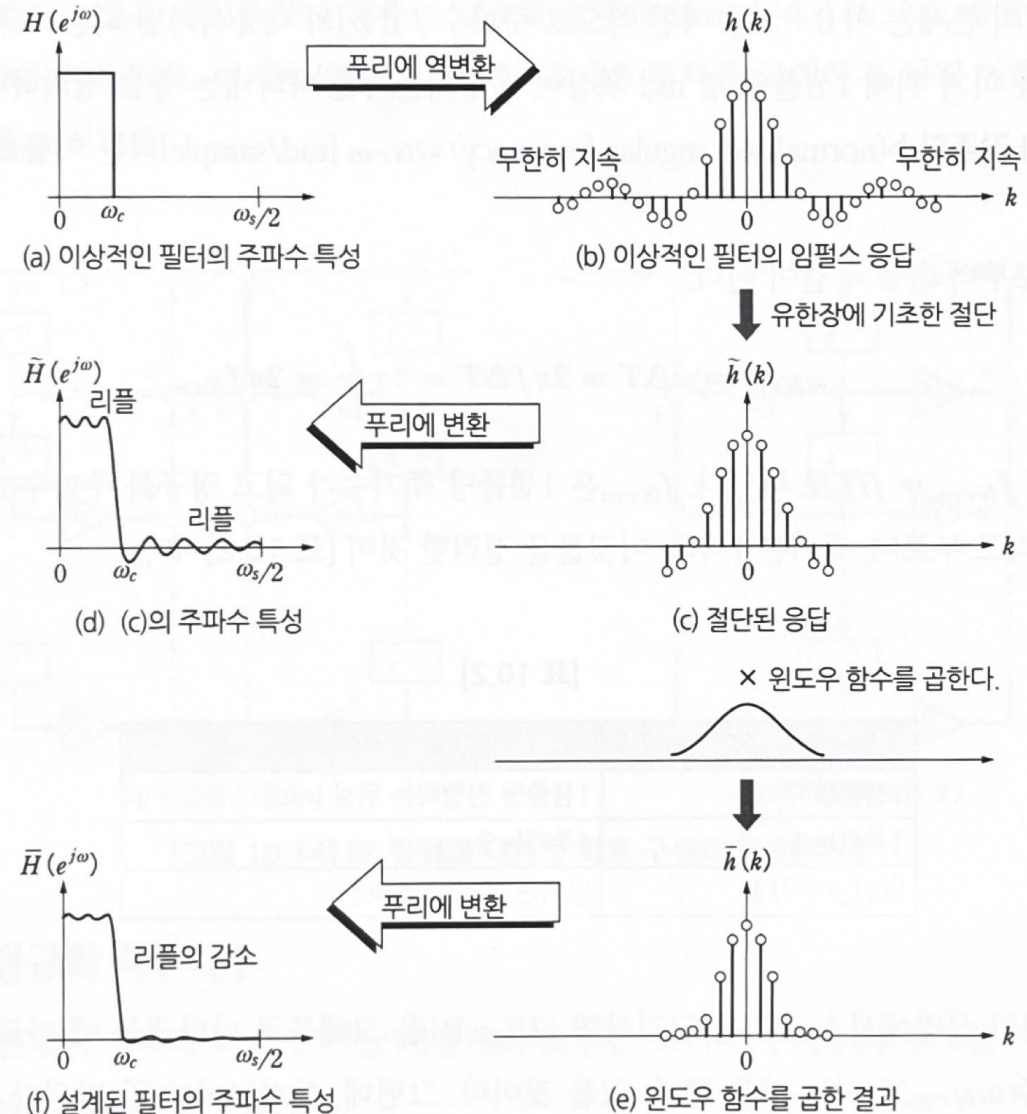
Physiological signal filtering



Things to consider: Phase shifting

원도우 함수를 이용한 FIR 설계

- 이상적인 low-pass filter에 해당하는 impulse function을 주파수 영역에서 추출하면 그림 (a)와 같음
- 이를 시간 영역으로 푸리에 역변환 하면 그림 (b)와 같이 무한히 지속되는 함수를 얻게 됨
- 그러나 컴퓨터 처리를 위해 유한장으로 절단하여 처리함 (그림 (c))
- 이것을 다시 푸리에 변환하여 주파수 영역으로 가져오면 그림 (d)와 같이 깃스현상*과 리플이 발생함
- 이 문제를 해결하기 위해 시간 영역의 impulse function에 window 함수를 곱하여 양 끝을 매끄럽게 절단함 (그림 (e))
- 이를 다시 주파수 영역으로 푸리에 변환 하면 리플이 감소된 것을 확인할 수 있음

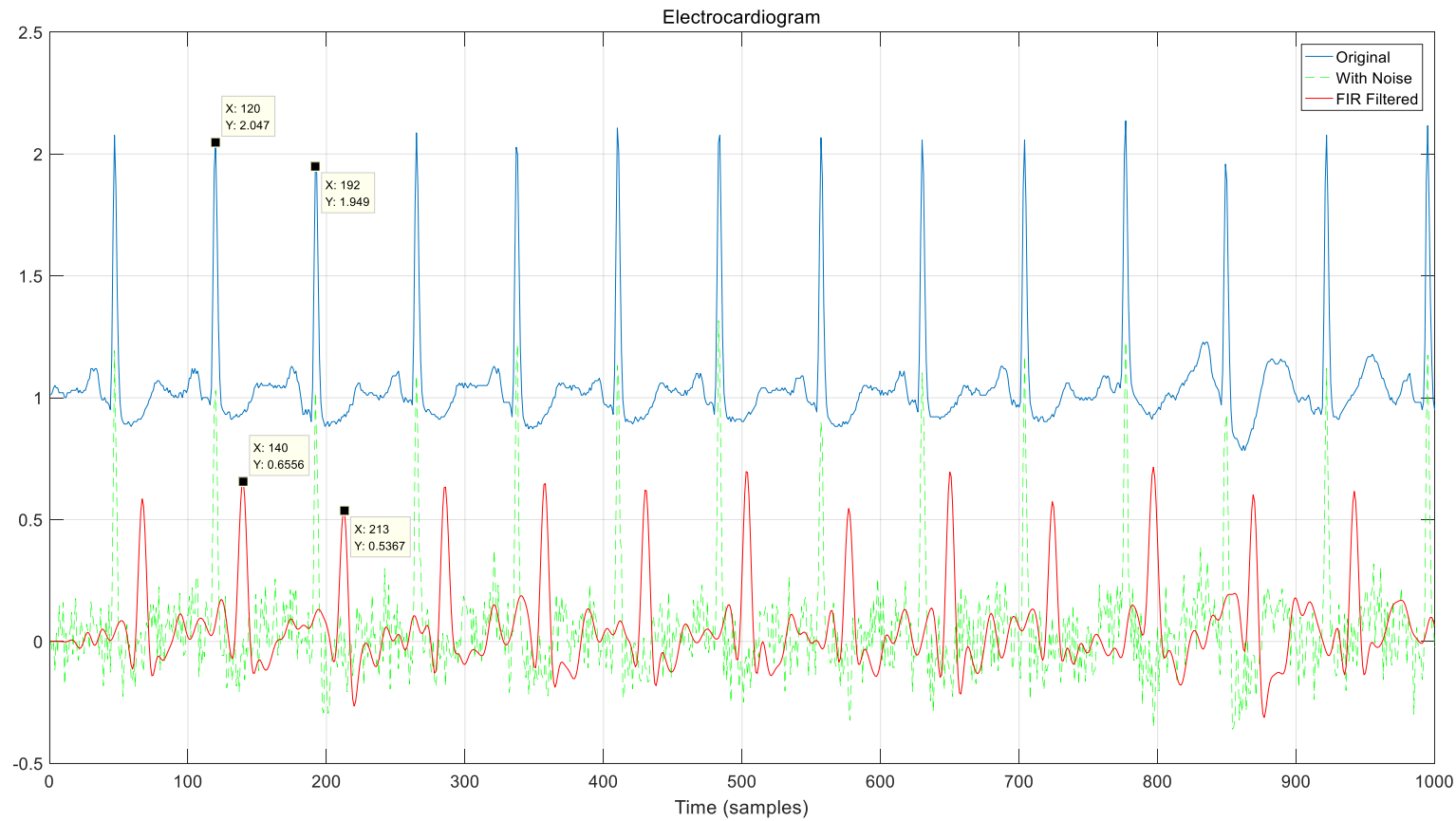


* 깃스현상: 불연속을 포함하는 파형이 푸리에 합성 되었을 때 불연속 값 근처에서 나타나는 불일치 현상. 푸리에 합성에서 고주파수 성분을 많이 포함할수록 불일치는 줄어들지만 유한한 주파수 대역에서는 완전히 없어지지 않는음.

Things to consider: Phase shifting

● Phase shifting

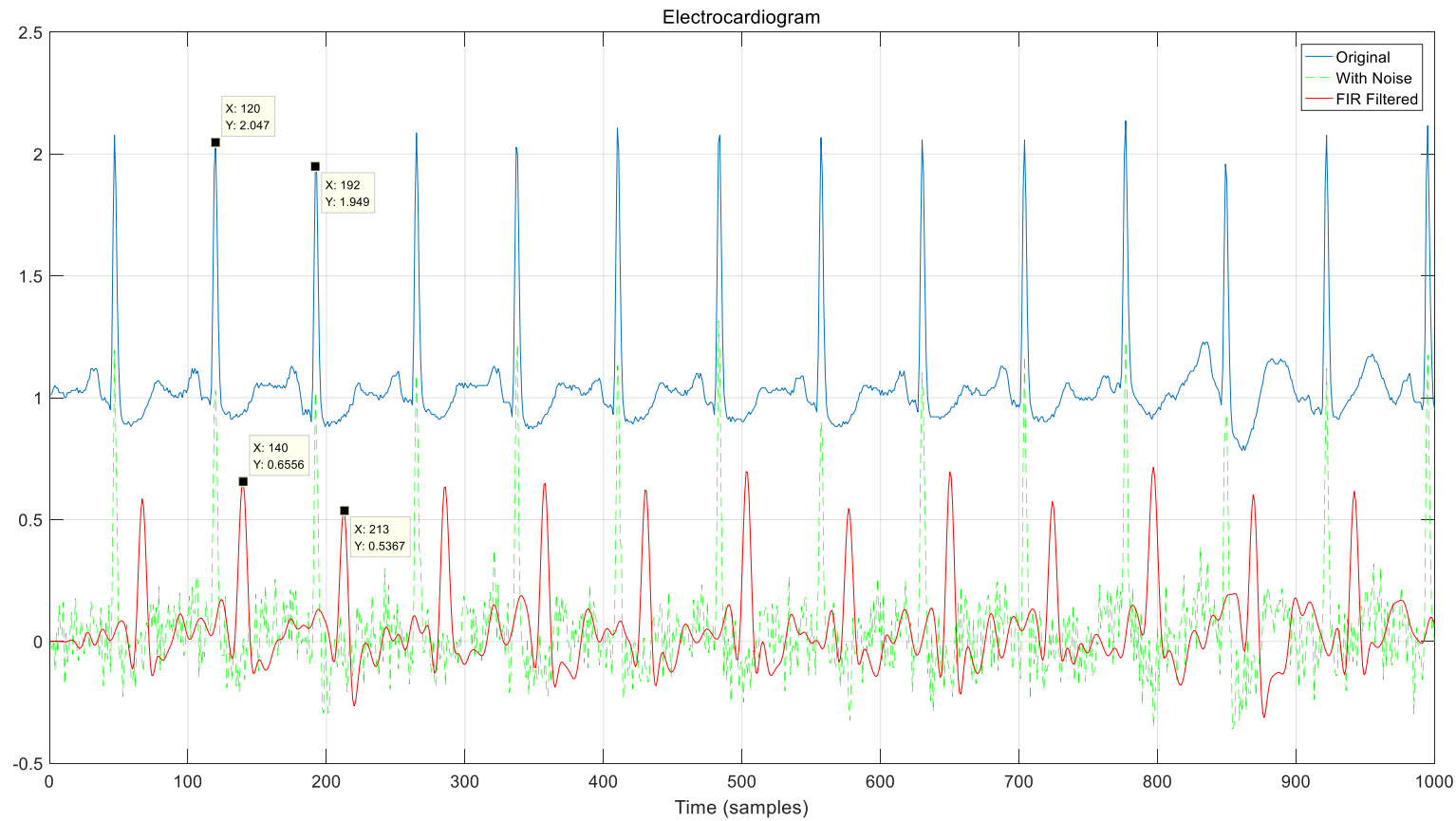
- 실제 impulse response 는 무한 임펄스 응답이지만 현실적인 적용을 위해 유한 임펄스 응답으로 비슷하게 함수를 구현하다 보니, 이 필터를 실제로 적용하기 위해서는 약간의 시간이 흐른 후의 “미래를 봐야만” 실제 값이 반영되어 나타난다.
- 이 때, 응답 신호의 지연은 위상 지연으로 나타나게 된다.
- 근사치의 정확도를 높이기 위해서는 지연도 더 길어져야 한다.



Things to consider: Phase shifting

● Phase shifting

- 실제 impulse response 는 무한 임펄스 응답이지만 현실적인 적용을 위해 유한 임펄스 응답으로 비슷하게 함수를 구현하다 보니, 이 필터를 실제로 적용하기 위해서는 약간의 시간이 흐른 후의 “미래를 봐야만” 실제 값이 반영되어 나타난다.
- 이 때, 응답 신호의 지연은 위상 지연으로 나타나게 된다.
- 근사치의 정확도를 높이기 위해서는 지연도 더 길어져야 한다.



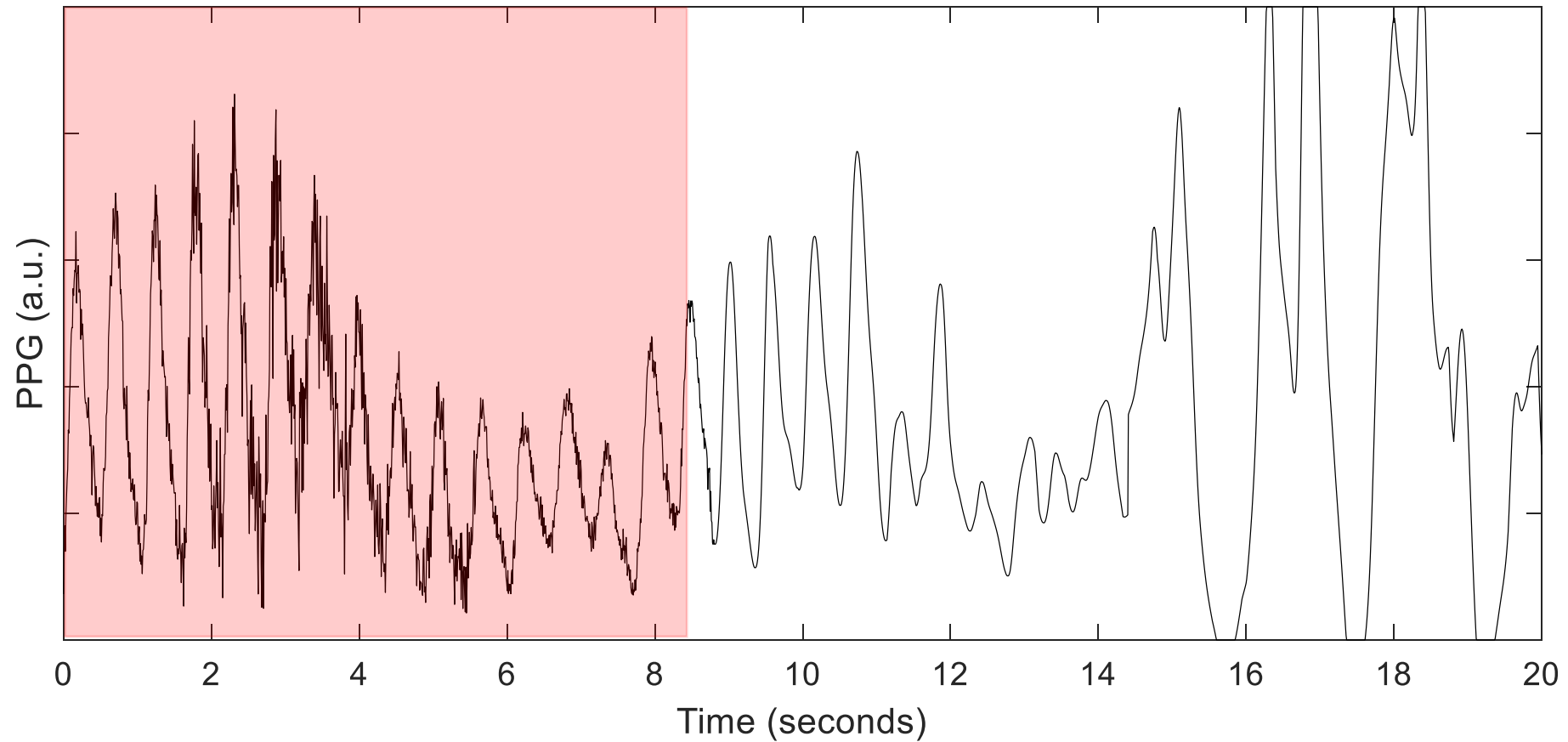
Source of Noise

*White noise &
low SNR*

*Saturation &
flat-lining*

*Low frequency
noise*

*Motion
artefacts*



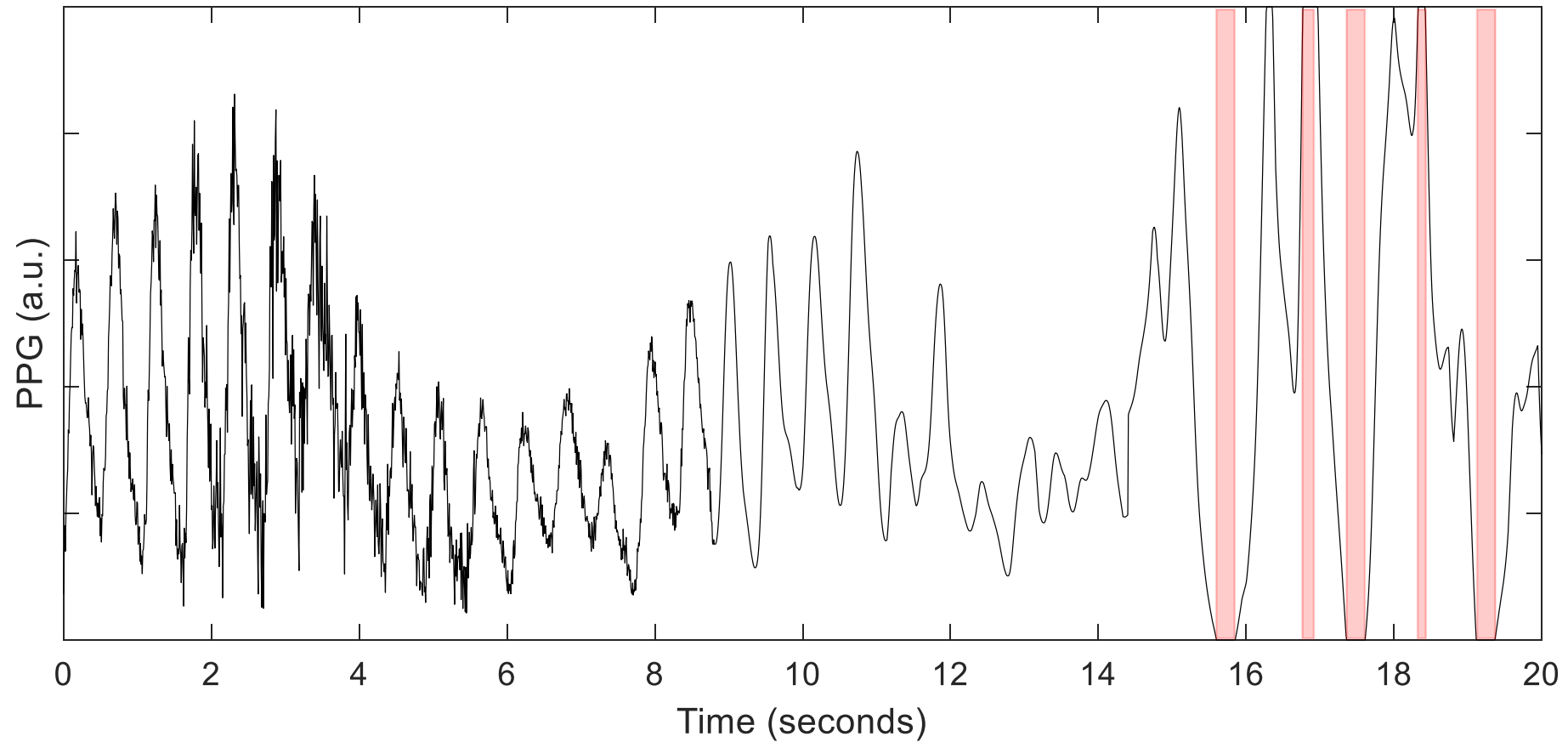
Source of Noise

*White noise &
low SNR*

*Saturation &
flat-lining*

*Low frequency
noise*

*Motion
artefacts*



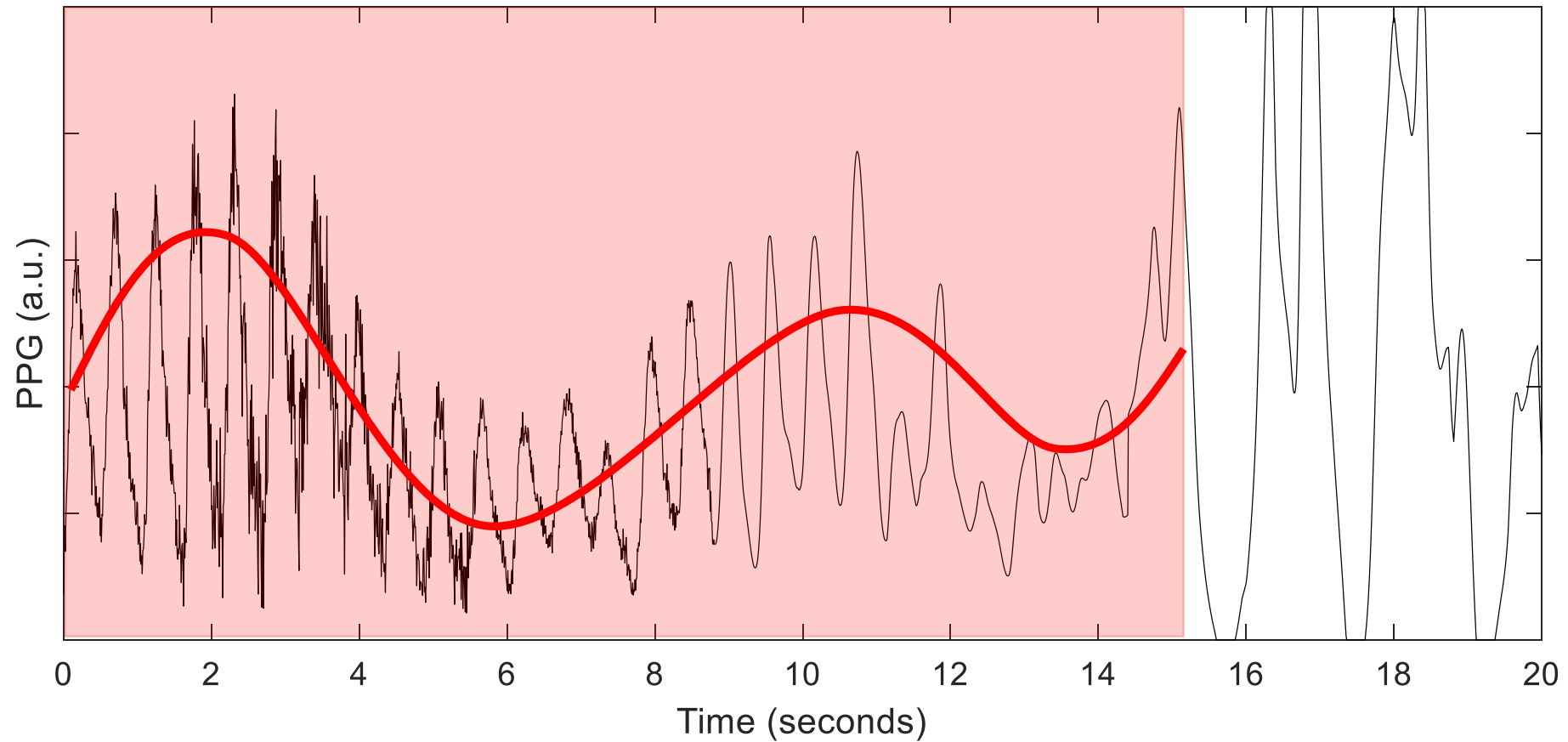
Source of Noise

*White noise &
low SNR*

*Saturation &
flat-lining*

*Low frequency
noise*

*Motion
artefacts*



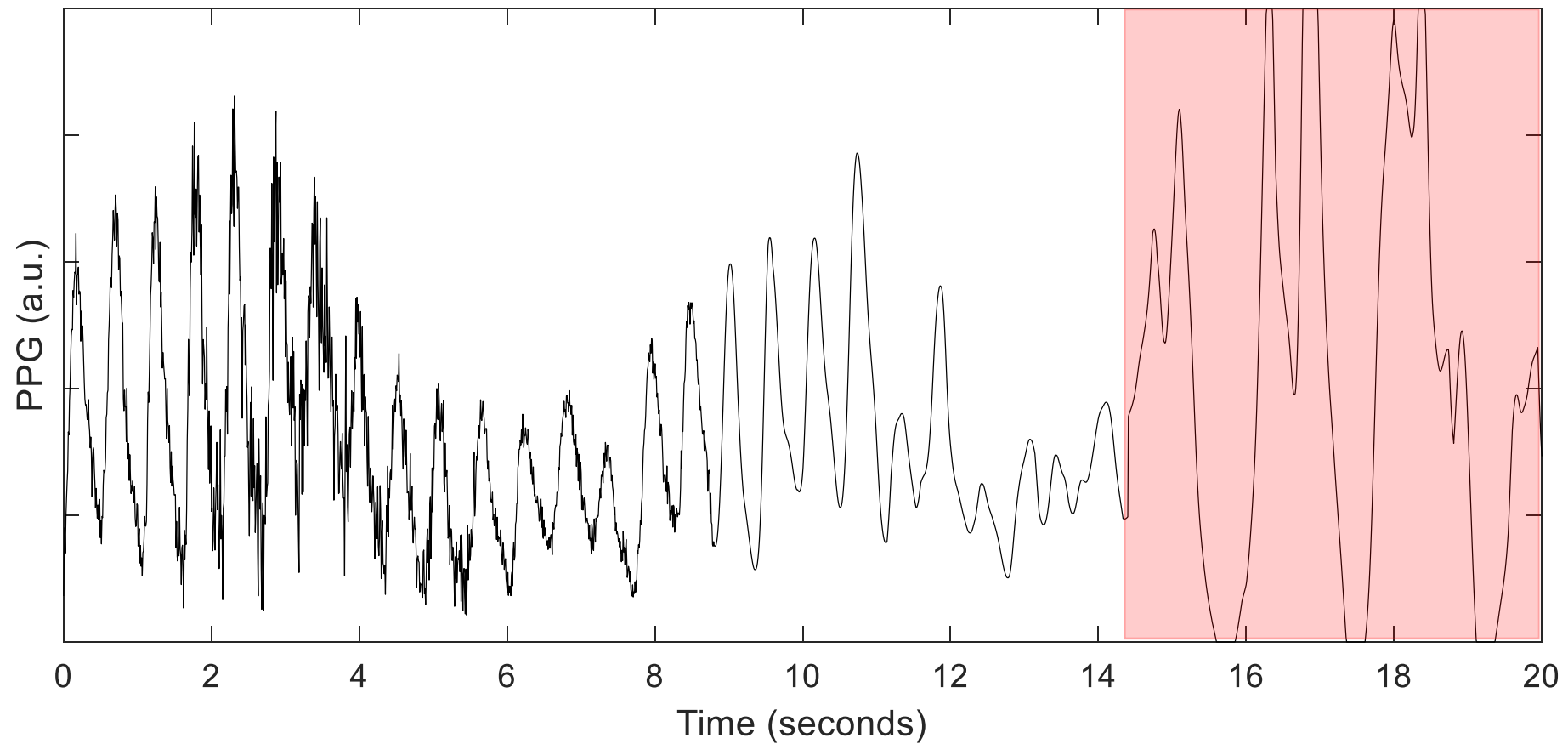
Source of Noise

*White noise &
low SNR*

*Saturation &
flat-lining*

*Low frequency
noise*

***Motion
artefacts***



- 목차
 - 신호 필터링
 - Source of Noise
 - Bandpass filtering
 - Removal of baseline wandering
 - 기타 고급 전처리 방법

Bandpass filter design using Scipy

100 sampling frequency
0.2~30Hz bandpass



```
sampling_rate = 100  
lowcut = 0.2  
highcut = 30
```

Calculate filter coefficients



Linear filtering using coefficients



```
# define butterworth filter  
def butter_bandpass(data, lowcut, highcut, sampling_rate, order=4, plot_filter=False):  
    nyq = 0.5 * sampling_rate # nyquist frequency  
    wn_low = lowcut / nyq # Wn for highpass butterworth filter  
    wn_high = highcut / nyq # Wn for lowpass butterworth filter  
    # note that, Wn must be normalised to 0,1  
  
    # calculate filter coefficients  
    b, a = butter(order, [wn_low, wn_high], btype='band')  
  
    # filter using linear filter function  
    y = lfilter(b, a, data)  
  
    if plot_filter:  
        # plotting frequency response  
        plt.figure()  
        w, h = freqz(b, a, worN=2000)  
        plt.plot((sampling_rate * 0.5 / np.pi) * w, abs(h), label='Butter order = %d' % order)  
        plt.xlabel('Frequency (Hz)')  
        plt.ylabel('Gain')  
        plt.grid(True)  
        plt.legend(loc='best')  
  
    return y
```

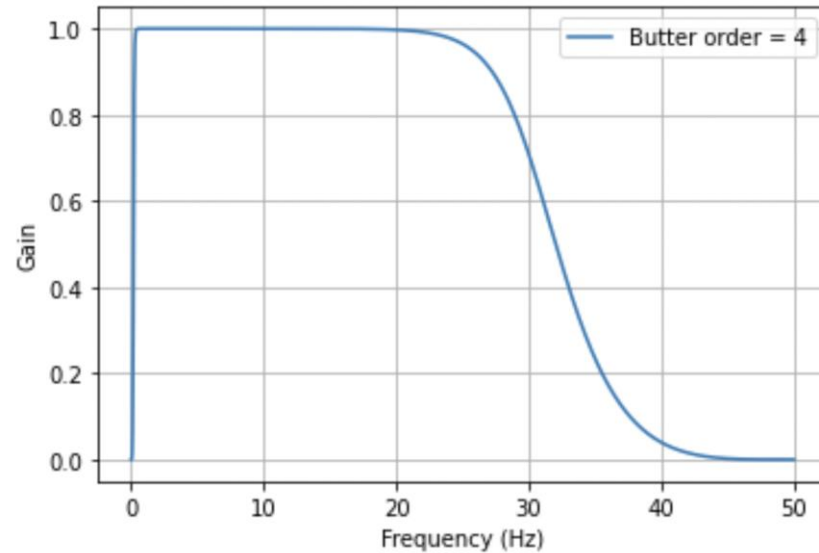
Bandpass filter function call



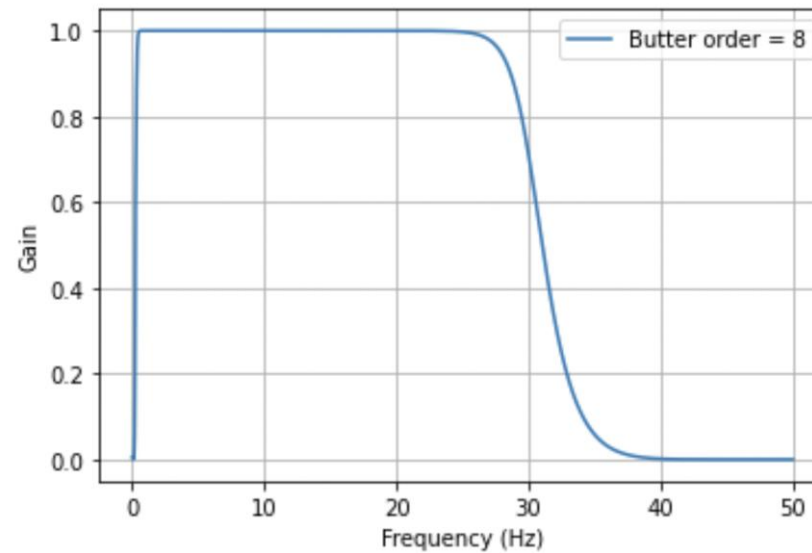
```
# filter the noisy ECG signal with butterworth filtering  
filtered_ECG_butter = butter_bandpass(ECG_data_noisy, lowcut, highcut, sampling_rate, plot_filter=True)  
fig = plt.figure(figsize=(14, 5))  
plt.plot(filtered_ECG_butter, color='b')  
plt.plot(ECG_data_noisy - 2, color='r')
```

Bandpass filter design using Scipy

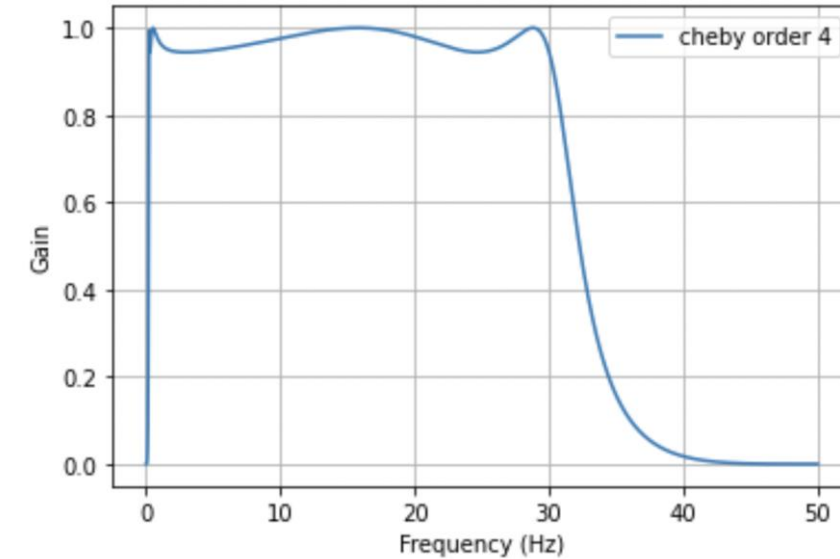
Butterworth filter
(order=4)



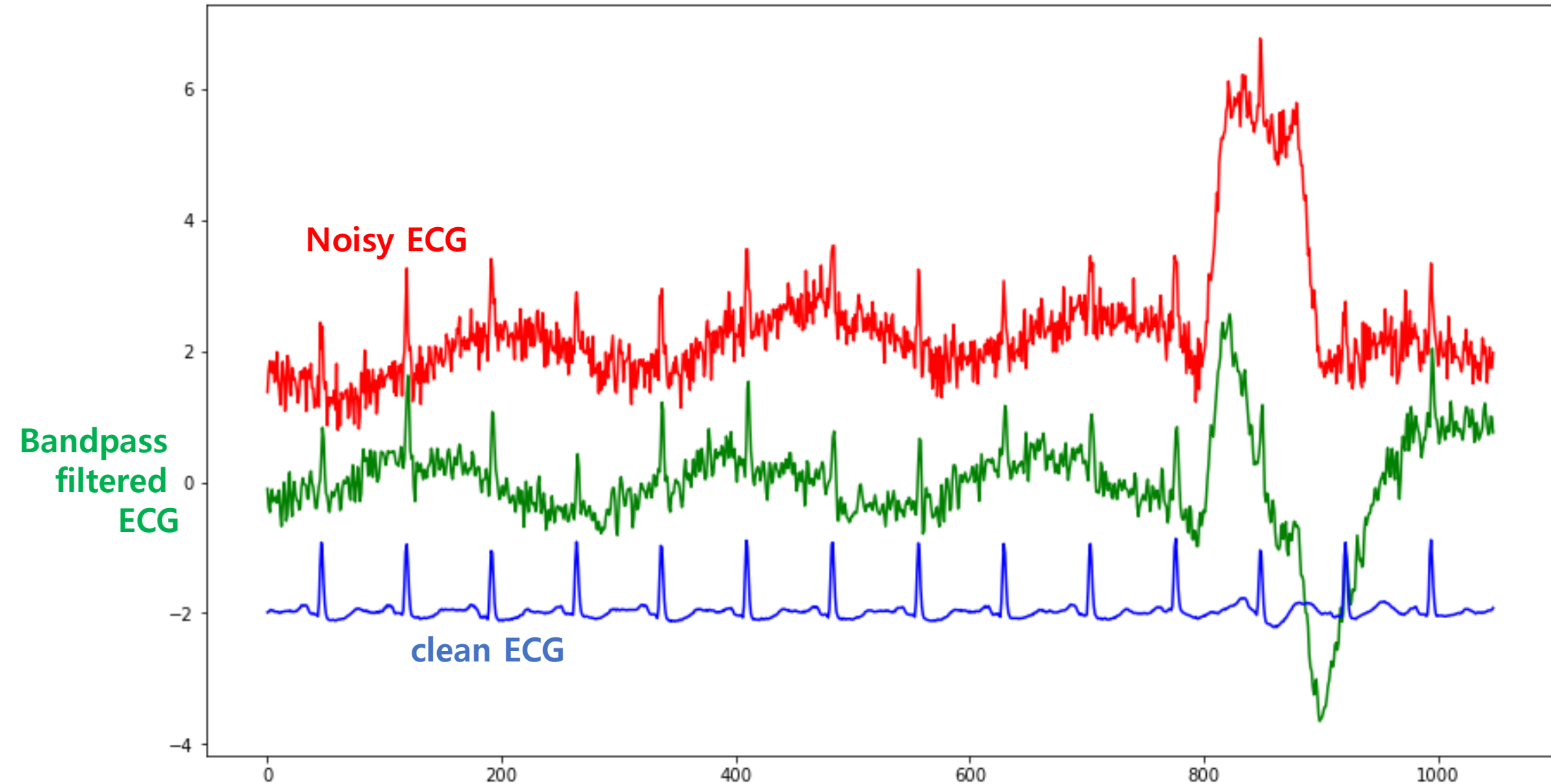
Butterworth filter
(order=8)



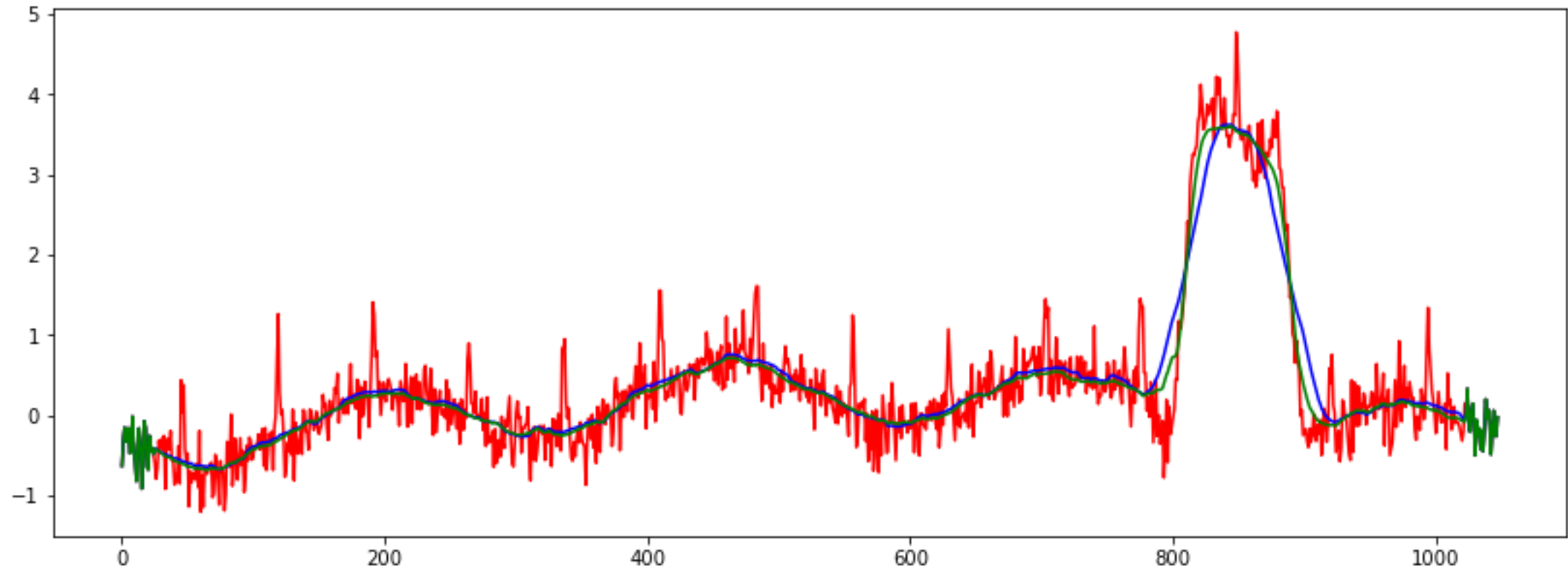
Chebyshev filter
(order=8)



Hands-on: Bandpass filtering

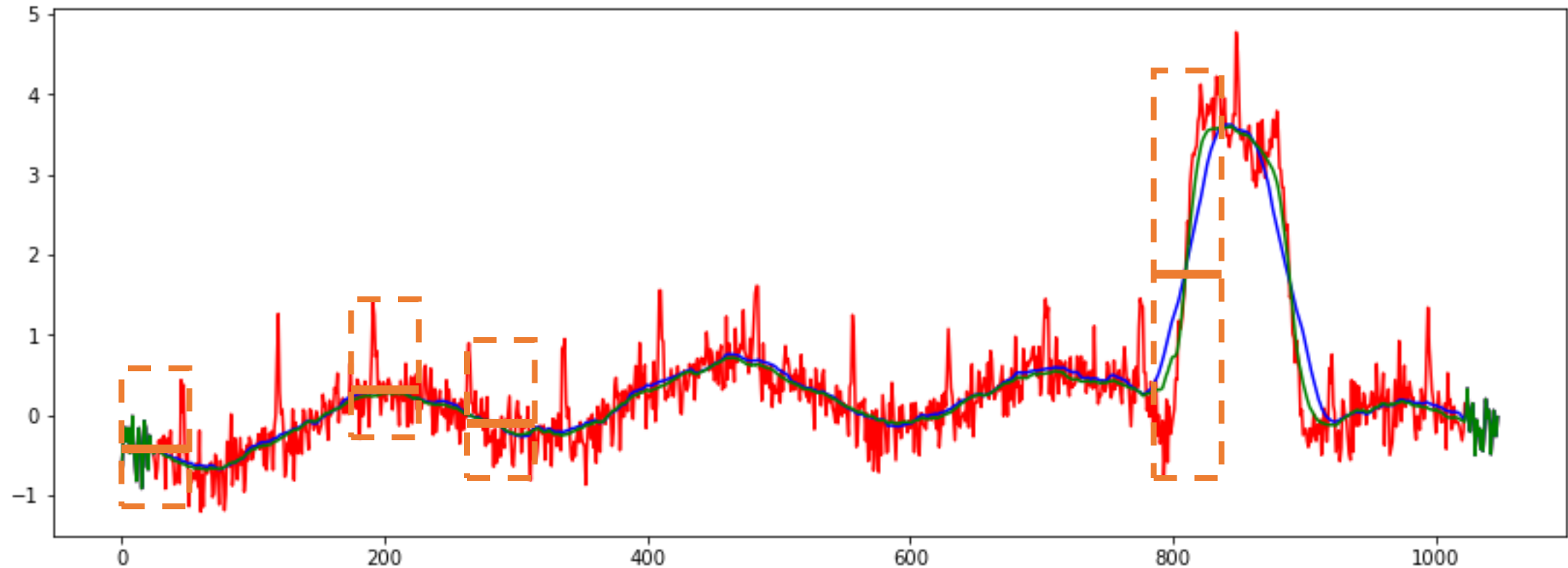


Removal of baseline wandering



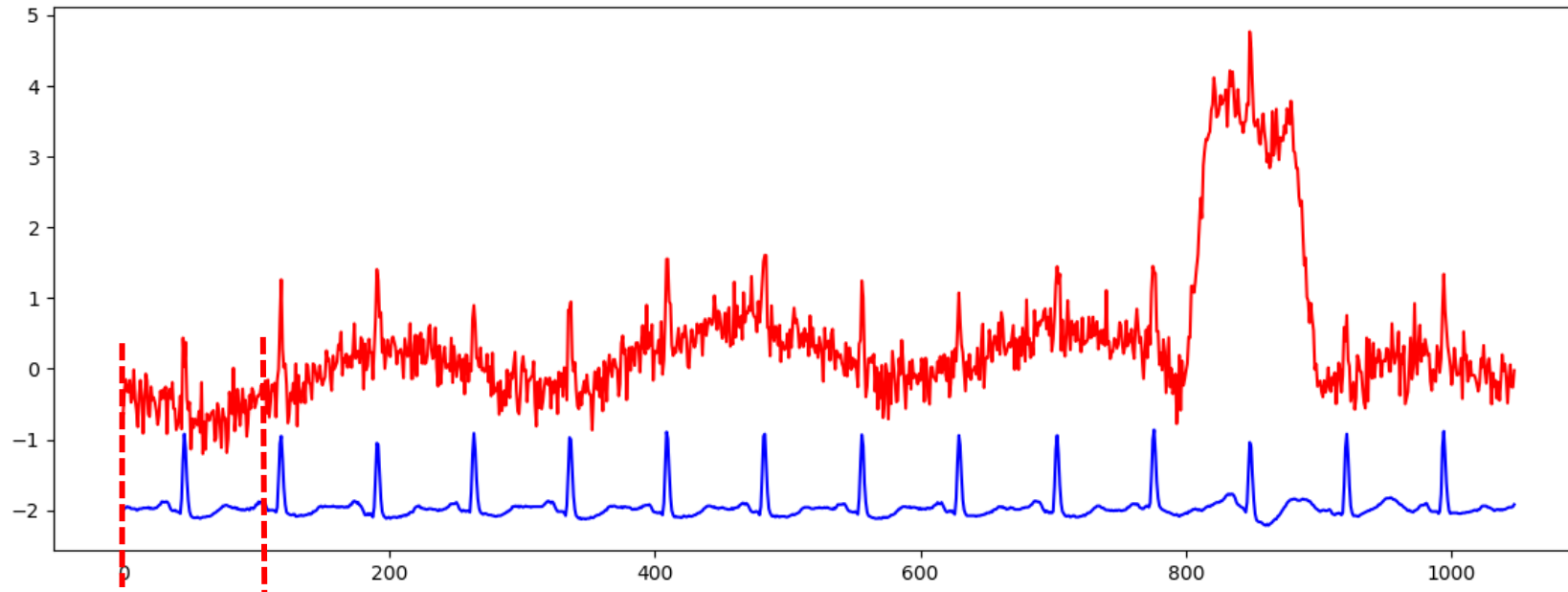
How do we get rid of the baseline fluctuations?

Removal of baseline wandering

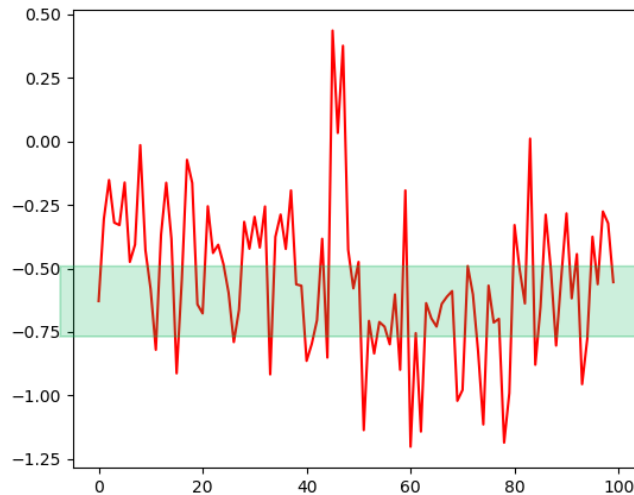


Subtract the noisy signal from the average of a specific window size, then repeat the process for every point in the recording!

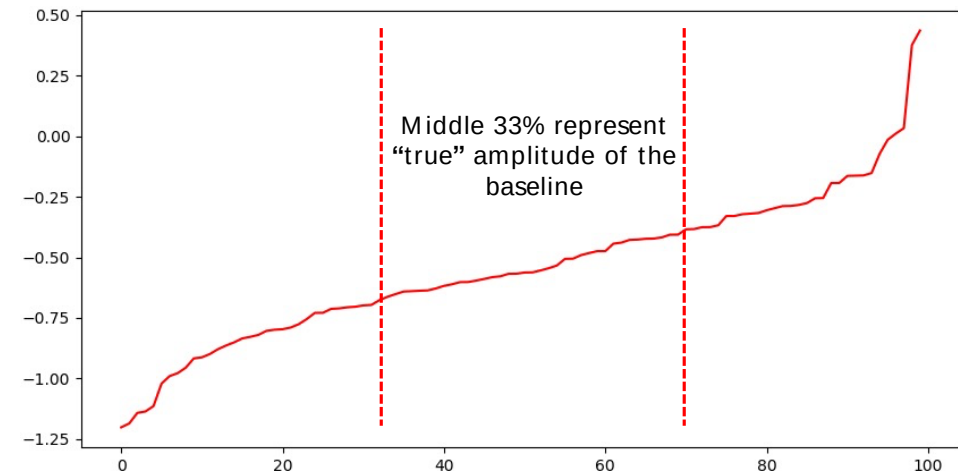
Non-linear baseline removal



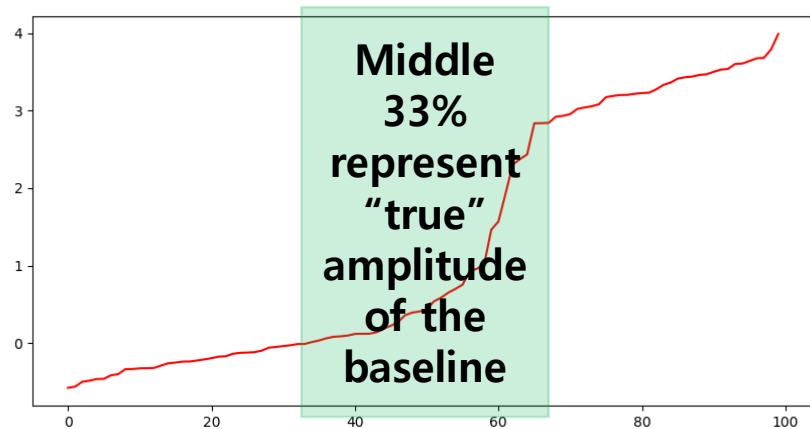
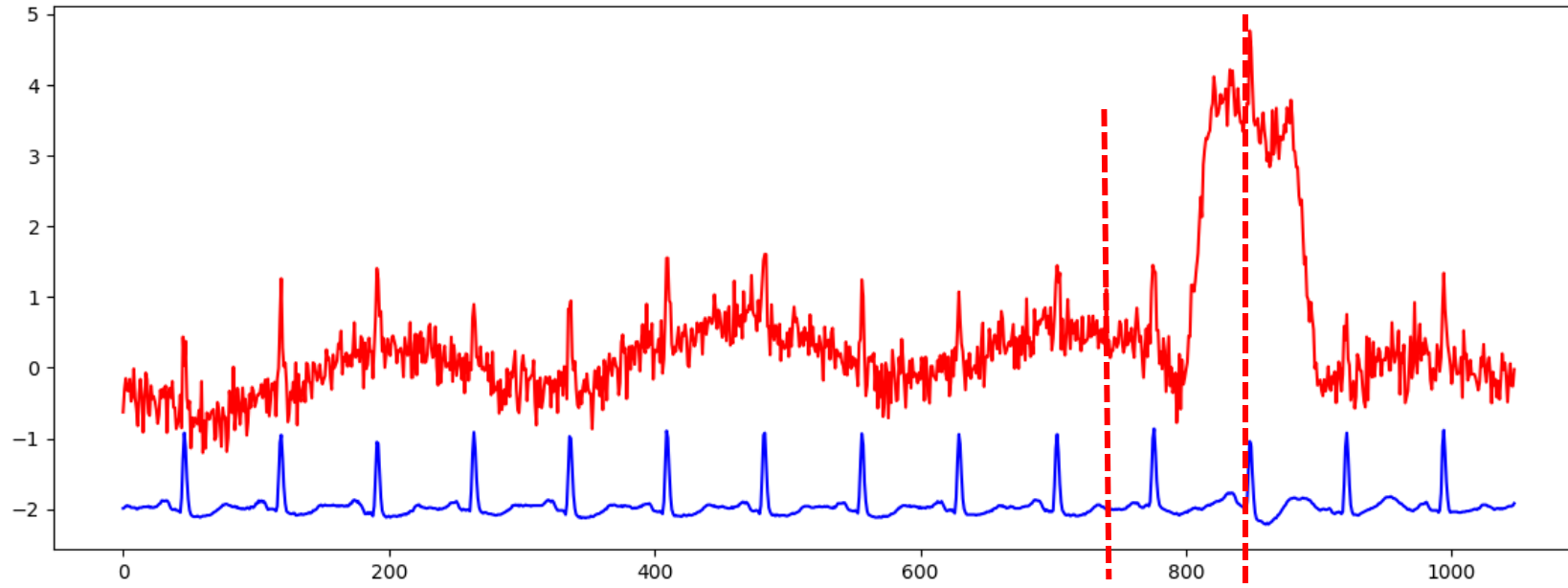
"true"
baseline



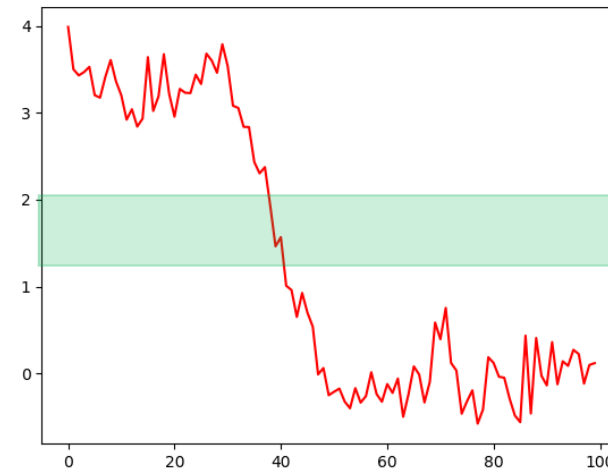
Sort by
amplitude



Non-linear baseline removal



Sort by
amplitude

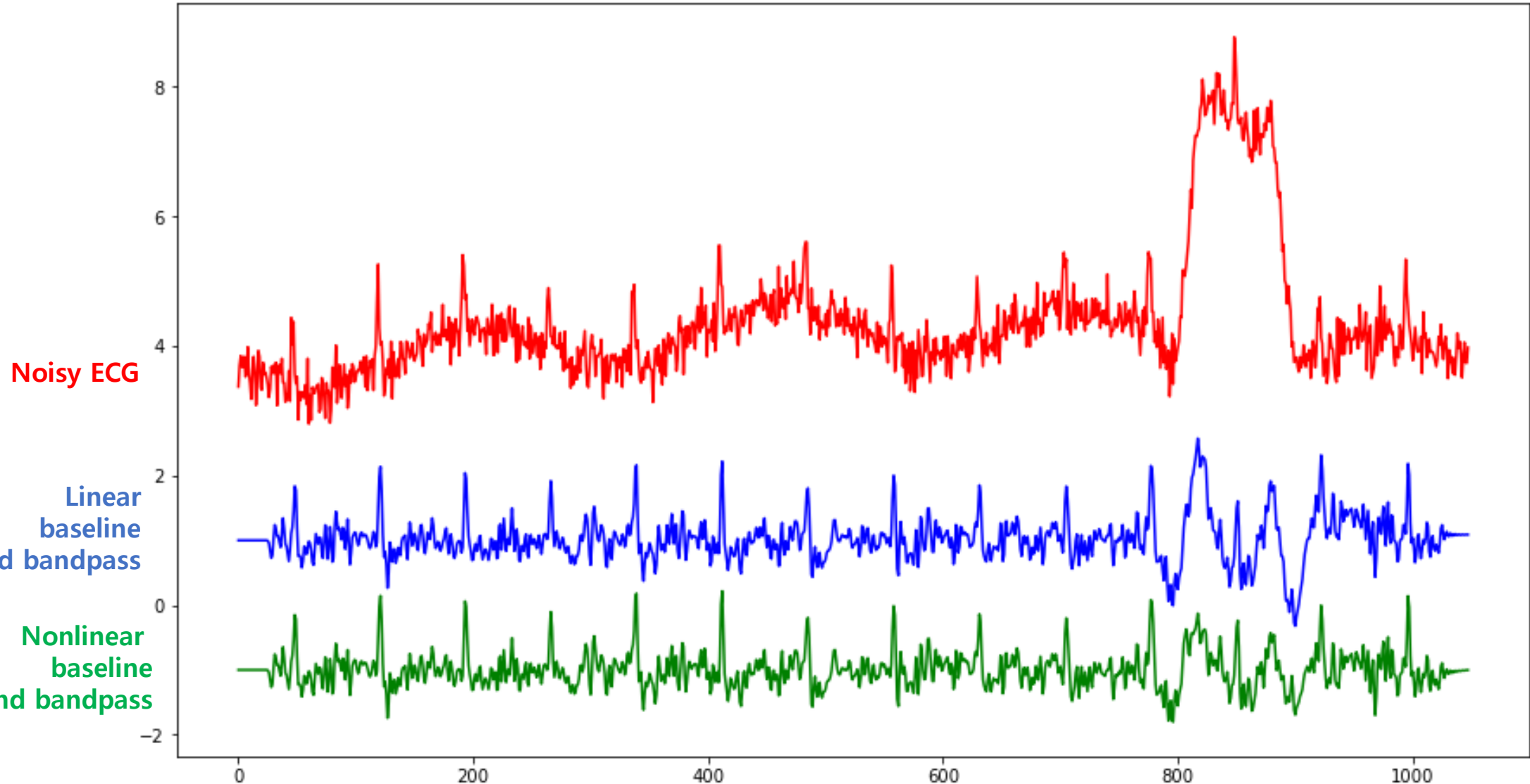


Removal of baseline wandering

```
# define linear baseline removal function
def linear_baseline(data, window):
    y = data.copy()
    start_point = int(window/2)
    end_point = int(len(data) - window/2)
    for i in range(start_point, end_point):
        y[i] = np.mean(data[i - window//2: i + window//2])
    return y
```

```
# define non-linear baseline removal function
def nonlinear_baseline(data, window):
    y = data.copy()
    start_point = int(window/2)
    end_point = int(len(data) - window/2)
    for i in range(start_point, end_point):
        sorted_window = np.sort(data[i - window//2 : i + window//2]) # sort the values in window
        y[i] = np.mean(sorted_window[window//3:2*window//3]) # use only 33% to 66% of data to find mean
    return y
```

Removal of baseline wandering



Other processing packages

